18-449/649 Lab 3: Dual-Redundant CAN and Real-Time Analysis

Misho Alexandrov (mvalexan), Ore Abolade (oabolade), T'sairus Beasley (tbeasley)

Repository: github.com/o-abolade/18-649-Tabletop-Drive-by-Wire-Car · code as of commit `7e944c4` (2026-10-24). The Lab 3 code is in `lab3_CAN/`.

System: N1 cockpit (Raspberry Pi 4), N2 steering and N3 drivetrain (Nucleo-F401RE, Zephyr), on two 500 kbit/s CAN buses. Every number is labeled **measured** (with the method) or **assumed** (with the reason). Scope captures are in Appendix A. The full log is in `lab3_CAN/Progress_doc_lab3.md`.

1. Wiring schematic

Bus layout

- **Order:** both buses are linear, N1 – N2 – N3.
- **Termination: P1 pads.** One 120 Ω at each end of each bus, from the module's P1 pad. **P1 is shorted on these four modules:**
  - N1 (Pi) CAN A and N1 CAN B;
  - N3 CAN A and N3 CAN B.

  N2's two modules have P1 open, and there are no discrete resistors on the rails, so each bus has exactly two terminators. Bus A measured 58 Ω CANH–CANL with power off.
- **Jumpers:** each CANH and CANL tap has its own jumper, which is the R4.8 fault-injection point.
- **Trade-off:** P1 sits behind the jumper, so pulling one of those four modules' jumpers also removes that end's terminator and leaves the bus with 120 Ω. Our pull tests on N1 and N3 still worked, but the margin is lower.

Pins

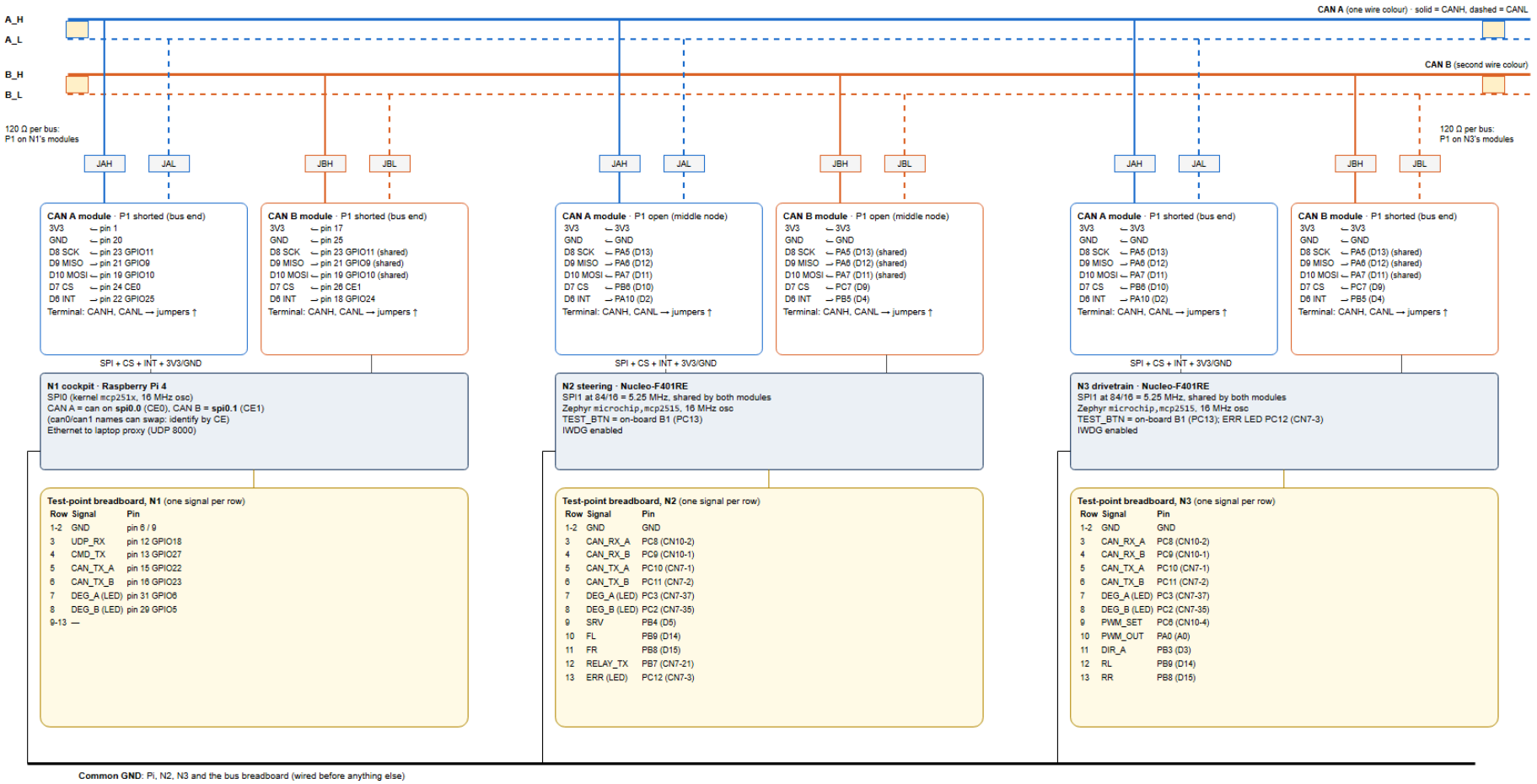
- **STM32:** SPI1 is shared (PA5/PA6/PA7). CS is PB6 (A) and PC7 (B); INT is PA10 (A) and PB5 (B).
- **Pi:** SPI0 CE0 (A) and CE1 (B). The cockpit picks each bus by chip select, because `can0` and `can1` swap between boots.
- **Module pinout:** D9 is MISO and D10 is MOSI; the handout figure has them swapped.

Part 0 and 1, measured

- **Crystal:** 16 MHz.
- **SPI clock:** 5.25 MHz. The next step, 10.5 MHz, exceeds the MCP2515's 10 MHz limit.
- **One frame read:** 21 µs on the wire, but one 3-byte register read through Zephyr's polled SPI driver takes 30 µs.
- **Zephyr's INT path:** the ISR only gives a semaphore. The SPI reads and our dispatch run in the driver's thread at `K_PRIO_COOP(2)`.
- **Part 1 checkpoint:** `candump` saw both Nucleos on both interfaces, and a pulled bus A jumper left bus B running.
- **Bus levels:** idle about 2.2 V on both lines; dominant differential 2.07 V on A and 1.97 V on B (photos 8–9).

Lab 3 wiring: dual-redundant CAN (CAN A, CAN B), 500 kbit/s, three nodes

Bus order on both buses: N1 (Pi) at one end, N2 in the middle, N3 at the other end. Every module is an XIAO CAN board (MCP2515 + SN65HVD230, 16 MHz crystal, 3.3 V). Each CANH/CANL tap goes through its own jumper wire = the R4.8 fault-injection point. Bus breadboard probe rows A_H, A_L, B_H, B_L. Common GND between Pi, both Nucleos and the bus breadboard.



Notes. (1) Module pin D9 is MISO and D10 is MOSI on these boards (the handout figure has them swapped). (2) Termination uses the P1 pad on N1's and N3's modules; P1 sits on the module side of the jumper, so pulling an end node's jumper also removes that bus end's terminator (R4.3 wants it on the bus side; known trade-off, discrete 120 Ω on the rail ends would fix it). (3) CANH–CANL measured powered off: bus A 58 Ω. (4) Bus levels (photos 8–9): idle ≈ 2.2 V both lines, dominant differential 2.07 V (A) / 1.97 V (B).

Figure 1: Dual-CAN wiring. Both buses with termination and jumpers, every module's SPI/CS/INT pins, and the three test-point boards

2. Task communication graph

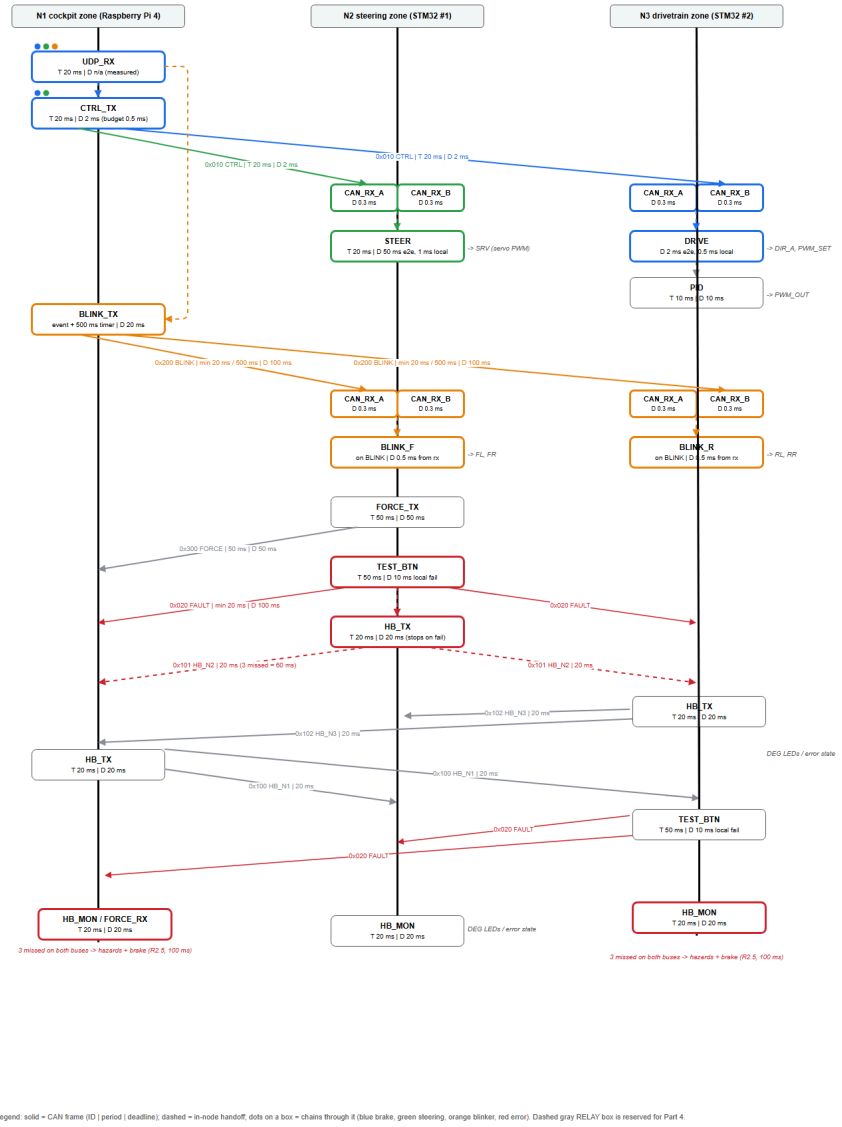


Figure 2: Task communication graph, one 20 ms cycle. CAN edges labeled ID, period and deadline; brake/throttle (blue), steering (green), blinker (orange) and error (red) chains highlighted

- **Edges:** solid edges are CAN frames; dashed edges are hand-offs inside one node (semaphores, or state under a spinlock or mutex).
- **Priorities:** the implemented ones are in 3.1. They include the per-bus CAN_TX threads and the MCP2515 driver threads, which the template folds into CAN_RX.

- **Part 4:** Figure 3 adds N2's RELAY task and the second bus hop.

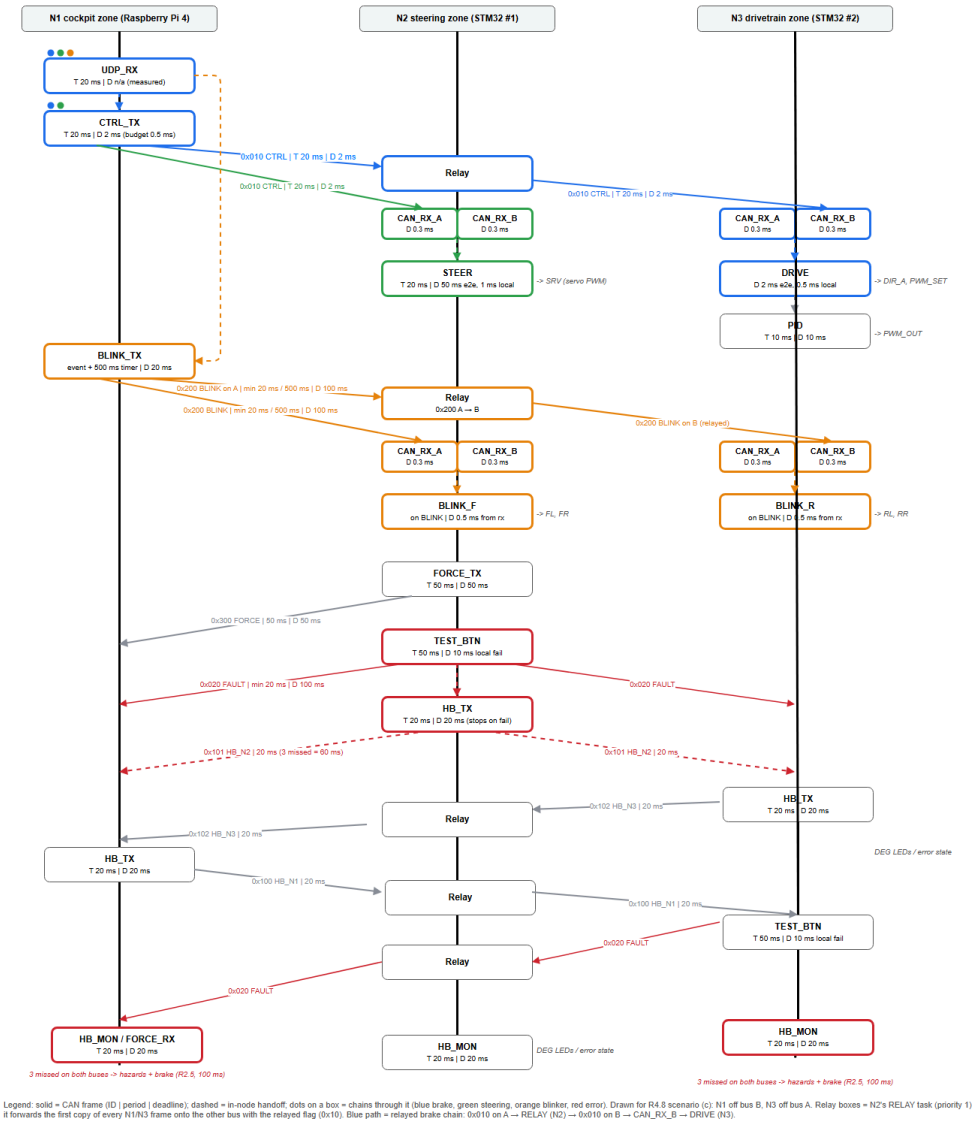


Figure 3: Extended task graph for scenario (c): RELAY on N2; relayed brake path 0x010 on A → RELAY → 0x010 on B → DRIVE

3. Single-bus real-time analysis

Following the TA clarification:

- **Scope:** one bus, with no redundancy and no relay (those are Section 4).
- **C values:** every C is the handout's Part 2 placeholder, except the four measured values below.
- **Priorities:** our implemented ones (`common/zephyr/zone_config.h`).
  - CAN_RX is the MCP2515 driver's cooperative thread, so it outranks every application task.
  - Our per-bus CAN_TX threads aren't separate rows. Their SPI loads are inside the template's HB_TX and FORCE_TX C (120 µs, both buses).
- **B = #4, one SPI frame load,** for every task except the lowest priority (B = 0). It isn't 2 × #4 because our send helper takes the SPI lock per bus: each bus has its own TX thread, and the lock is held for one transaction.
- **R** = C + B + Σ over higher-priority j of ⌈R/T_j⌉·C_j, iterated until it stops changing.
  - Every R here is far below 10 ms, so each ⌈R/T_j⌉ is 1 and each R is a single sum.
  - CAN_RX runs once per received message: 7 messages per 20 ms on one bus, so it interferes 7 times.

3.1 Measurements

Measured on the scope, maximum over at least a minute of traffic, with the node's other tasks idle. The test points are C GPIO writes, a few µs each. Screenshots are in Appendix A.1.

#	Number	Start probe → stop probe	Max (µs)	Screenshot
1	Pi stage	N1 UDP_RX (after the socket read) → CMD_TX (after the CAN writes)	126.5	M1
2	CAN_RX C, N3	N3 CAN A INT (PA10 / D2) falling → CAN_RX_A	309	M2
3	DRIVE C, N3	N3 CAN_RX_A → DIR_A on a brake frame, N3's bus B unplugged (single bus, as Part 3 assumes). With both buses connected the max was 503 µs: the CTRL copy from bus B arrives right after and N3's bus B driver thread preempts DRIVE for about 300 µs (M2)	147	M3
4	SPI frame load (B)	MCP2515 CS (PB6 / D10) low → high, longest window	116.5	M4

3.2 Node RTA: N3 (drivetrain)

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ≤ D?	Work: C + B + higher-priority C
CAN_RX	coop 2	309	20 ms per message, 7 per bus	0.3 ms	116.5	425.5	**No**	309 + 116.5 = **425.5**
DRIVE	2	147	20 ms	0.5 ms	116.5	2,426.5	**No**	147 + 116.5 + 7·309 = **2,426.5**
BLINK_R	3	20	20 ms	0.5 ms	116.5	2,446.5	**No**	20 + 116.5 + 7·309 + 147 = **2,446.5**
TEST_BTN	4	20	50 ms	10 ms	116.5	2,466.5	Yes	20 + 116.5 + 7·309 + 147 + 20 = **2,466.5**
PID	5	200	10 ms	10 ms	116.5	2,666.5	Yes	200 + 116.5 + 7·309 + 147 + 20 + 20 = **2,666.5**
HB_MON	6	30	20 ms	20 ms	116.5	2,696.5	Yes	30 + 116.5 + 7·309 + 147 + 20 + 20 + 200 = **2,696.5**

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ≤ D?	Work: C + B + higher-priority C
HB_TX	7	120	20 ms	20 ms	0 (lowest)	2,700	Yes	120 + 0 + 7·309 + 147 + 20 + 20 + 200 + 30 = **2,700**

3.3 Node RTA: N2 (steering)

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ≤ D?	Work: C + B + higher-priority C
CAN_RX	coop 2	80	20 ms per message, 7 per bus	0.3 ms	116.5	196.5	Yes	80 + 116.5 = **196.5**
STEER	2	50	20 ms	1 ms	116.5	726.5	Yes	50 + 116.5 + 7·80 = **726.5**
BLINK_F	3	20	20 ms	0.5 ms	116.5	746.5	**No**	20 + 116.5 + 7·80 + 50 = **746.5**
TEST_BTN	4	20	50 ms	10 ms	116.5	766.5	Yes	20 + 116.5 + 7·80 + 50 + 20 = **766.5**
HB_MON	6	30	20 ms	20 ms	116.5	796.5	Yes	30 + 116.5 + 7·80 + 50 + 20 + 20 = **796.5**
HB_TX	7	120	20 ms	20 ms	116.5	916.5	Yes	120 + 116.5 + 7·80 + 50 + 20 + 20 + 30 = **916.5**
FORCE_TX	8	120	50 ms	50 ms	0 (lowest)	920	Yes	120 + 0 + 7·80 + 50 + 20 + 20 + 30 + 120 = **920**

Any task marked No above misses for the same reason: CAN_RX outranks every application task and is charged 7 times, once per catalog message per 20 ms. This is the template's conservative assumption that all 7 messages arrive inside the window. One bus can deliver at most ⌈R / 270 µs⌉ frames in a window of length R, so that tighter bound would remove most misses. Other fixes: a shorter receive path (lower CAN_RX C), or priority above CAN_RX dispatch for the tight 0.5 ms tasks.

3.4 Message RTA: CTRL (same on both buses)

Message	J (µs)	C (µs)	B (µs)	w (µs)	R (µs)
0x010 CTRL	126.5 (#1)	270	270	270	**666.5**

- **w:** CTRL has the lowest ID, so there is no higher-priority term, and w is just the 270 µs blocking from one lower-priority frame already on the wire.
- **R** = J + w + C = 126.5 + 270 + 270 = 666.5 µs. "CTRL on bus" in the chains is w + C = 540 µs.
- **BLINK** (used for the blinker start) has 5 higher-priority IDs (CTRL, FAULT, HB_N1–3), each at most once in the window: w = 270 + 5·270 = 1,620 µs, so BLINK on bus = w + C = 1,890 µs.

3.5 End-to-end chains (single bus)

Chain	Stages to add up (µs)	Total	Deadline	Met?
Brake / throttle	Pi 126.5 + CTRL on bus 540 + CAN_RX (N3) 425.5 + DRIVE 2,426.5	3,518.5	2 ms	**Missed.** DRIVE carries 7 CAN_RX interferences on top of its own receive path. Fix: a tighter arrival bound or a shorter receive path (Appendix B).
Steering	Pi 126.5 + CTRL on bus 540 + CAN_RX (N2) 196.5 + STEER 726.5 + one servo period 20,000	21,589.5	50 ms	**Met**
Blinker start	Pi 126.5 + BLINK on bus 1,890 + CAN_RX (N3) 425.5 + BLINK_R 2,446.5	4,888.5	100 ms	**Met**
Blinker sync	Worst case of one receive path minus the other's best case (C only): N3 2,872 − N2 100, and N2 943 − N3 329	2,772	1 ms	**Missed.** The two zones see different interference after the same frame. Fix: lower CAN_RX C, or a tighter arrival bound.

Chain	Stages to add up (µs)	Total	Deadline	Met?
Error propagation	TEST_BTN (N2) 766.5 + 3 missed heartbeats 66,000 + one HB_MON period 20,000 + HB_MON (N3) 2,696.5	89,463	100 ms	**Met**

- **Blinker sync:** both STM32s receive the same BLINK frame at the same instant from the same bus. So the edge offset is only the difference between their receive-to-lamp paths. The bound is the larger of N3's worst case minus N2's best case and the reverse, so it is symmetric by construction.
- **Error propagation:** this is the heartbeat path. Our zones also send FAULT on a test-button press, which reached the other zone in 1.1 ms in the lab (photo 13).

4. Dual-bus and relay analysis

4.1 Hot/parallel transmit

- **Send:** every message goes on bus A and then bus B with one sequence number (`can_link_send`).
- **Receive:** one dispatch path keeps the first copy by `(ID, seq)` and drops the second; FAULT has one dedup slot per sending zone.
- **Cost on the receiver:** each message now costs two receive interrupts and two SPI reads, so CAN_RX runs 14 times per 20 ms instead of 7 (the N2 table below).
- **The chains get no worse,** because each receiver acts on whichever copy arrives first and the duplicate only adds interference. That assumes the two copies leave together, which they do: the Pi writes A then B inside the measured #1 stage.

4.2 Relay design

Question	Our decision	Why
Who relays?	**N2 and N3**, the two STM32s; never the Pi	The checkoff's four cases need N2 (N1/N3 on opposite buses) or N3 (N1/N2 on opposite buses). A relay on the Pi would put Linux, which has no worst-case bound, in the path.
When?	**Always**, with no fault detection	A relay that waits for the 60 ms degraded detection would cut CTRL (brakes included) for 60–70 ms after the fault. Always relaying costs bus time instead (4.5).
What?	The **first copy** of every frame the zone didn't send itself, onto the other bus	A node already sends its own frames on both buses.
Loops?	**Dedup, no flag:** a zone relays only the *first* copy of each `(ID, seq)` it accepts, and never its own frames; the copy is forwarded **unchanged**	Each zone forwards a given frame at most once, so forwarding stops once every zone has it. The per-ID sequence state that drops duplicates is the only state needed. An earlier version set a "relayed" bit, but that made a relayed copy differ by one bit from the sender's own copy of the same ID: when both started arbitration together, both transmitters saw a bit error and retried into each other (TEC rose to ~76 on the bench). Identical frames sent together are legal CAN.
RELAY priority?	**1**: above STEER (2) and every application task; below the MCP2515 driver threads and the per-bus CAN_TX threads	RELAY is on the relayed brake path, so it must not wait behind STEER. The receive hook only queues the frame (16 deep) and wakes RELAY. TX queues are sorted by ID, so a relayed CTRL overtakes waiting HB/FORCE.
Cost to N2's own tasks	RELAY: 6 jobs per 20 ms in (c)	In the N2 table below.

Checkoff cases: N1 off B + N3 off A and N1 off A + N3 off B (N2 relays); N1 off A + N2 off B and N1 off B + N2 off A (N3 relays). We analyze the first, as the clarification asks.

4.3 Measurements

#	Number	Start probe → stop probe	Max (µs)	Screenshot
5	CAN_RX C, N2	N2 CAN A INT (PA10 / D2) falling → CAN_RX_A	289.5	M5
6	RELAY C, N2	N2 CAN_RX_A → RELAY_TX, pulsed right after the relayed frame's `can_send()` on bus B returns; N1 off B, N3 off A	405	M6

4.4 Re-analysis: N2 with RELAY

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ≤ D?	Work: C + B + higher-priority C
CAN_RX	coop 2	289.5	20 ms per message, 7 per bus × 2 buses	0.3 ms	116.5	406	**No**	289.5 + 116.5 = **406**
RELAY	1	405	every frame N2 forwards: 6 per 20 ms in (c)	on the brake chain	116.5	4,574.5	see chain	405 + 116.5 + 14·289.5 = **4,574.5**
STEER	2	50	20 ms	1 ms	116.5	6,649.5	**No**	50 + 116.5 + 14·289.5 + 6·405 = **6,649.5**
BLINK_F	3	20	20 ms	0.5 ms	116.5	6,669.5	**No**	20 + 116.5 + 14·289.5 + 6·405 + 50 = **6,669.5**
TEST_BTN	4	20	50 ms	10 ms	116.5	6,689.5	Yes	20 + 116.5 + 14·289.5 + 6·405 + 50 + 20 = **6,689.5**
HB_MON	6	30	20 ms	20 ms	116.5	6,719.5	Yes	30 + 116.5 + 14·289.5 + 6·405 + 50 + 20 + 20 = **6,719.5**
HB_TX	7	120	20 ms	20 ms	116.5	6,839.5	Yes	120 + 116.5 + 14·289.5 + 6·405 + 50 + 20 + 20 + 30 = **6,839.5**
FORCE_TX	8	120	50 ms	50 ms	0 (lowest)	6,843	Yes	120 + 0 + 14·289.5 + 6·405 + 50 + 20 + 20 + 30 + 120 = **6,843**

4.5 Bus utilization with our relay design

Starting from the template's 8.6 % (6 frames per 20 ms + FORCE per 50 ms), each frame relayed every 20 ms adds 270 / 20,000 = 1.35 %.

Bus	Relayed onto it in scenario (c)	New utilization	≤ 50 %?
A	HB_N3', FAULT_N3' (N3's frames, from B), each 20 ms: 8.6 + 2 × 1.35	**11.3 %**	Yes
B	CTRL', HB_N1', BLINK', FAULT_N1' (N1's frames, from A), each 20 ms: 8.6 + 4 × 1.35	**14.0 %**	Yes

Normal operation, with both buses up, is the worst case for always-relay:

- N2 forwards N1's 4 and N3's 2 frames per 20 ms.
- N3 forwards N1's 4 frames, plus HB_N2 and FAULT_N2 (20 ms) and FORCE (50 ms).
- If every copy lands on one bus: 8.6 + 12 × 1.35 + 0.54 = **25.3 %**, still under 50 %.

4.6 Relayed CTRL on bus B

Message	J (µs)	C (µs)	B (µs)	w (µs)	R (µs)
0x010 CTRL, relayed	Pi 126.5 + CTRL on A 540 + CAN_RX R (N2) 406 + RELAY R (N2) 4,574.5 = **5,647**	270	270	270	**6,187**

J is the whole first hop. On bus B the relayed CTRL still has the lowest ID, so w = 270 µs of blocking and R = J + 540. The relayed brake chain is this R + CAN_RX (N3) + DRIVE, so the first hop isn't counted twice.

4.7 Chains in scenario (c): N1 off bus B, N3 off bus A

Chain	Stages to add up (µs)	Total	Deadline	Met?	What changed
Brake / throttle	relayed CTRL R 6,187 (= Pi + CTRL on A + CAN_RX (N2) + RELAY + CTRL on B) + CAN_RX (N3) 425.5 + DRIVE 2,426.5	9,039	2 ms	**Missed.** The extra hop adds N2 receive + RELAY + a second bus traversal. Fix: the handout option we take: accept documented degraded latency in this double fault; or forward at the driver/ISR level.	Second bus hop through N2's RELAY

Chain	Stages to add up (µs)	Total	Deadline	Met?	What changed
Steering	Pi + CTRL on A 540 + CAN_RX (N2) 406 + STEER 6,649.5 + servo period 20,000	27,722	50 ms	**Met**	N2 still gets CTRL directly on A; only its own load grew (RELAY, 2 buses)
Blinker start	Pi + BLINK on A 1,890 + CAN_RX (N2) + RELAY + BLINK' on B 1,890 + CAN_RX (N3) + BLINK_R	11,759	100 ms	**Met**	N3 gets BLINK only through the relay
Blinker sync	N3's path through the relay: RELAY + BLINK' on B + CAN_RX (N3) + BLINK_R, minus N2's best case	9,236.5	1 ms	**Missed.** N3 can only act on the relayed copy, one full hop after N2. Fix: accept as degraded in (c), or have N2 delay its own edge by the relay latency while relaying.	N3's edge now trails N2's by the relay hop
Error propagation	TEST_BTN (N2) 6,689.5 + 66,000 + 20,000 + HB_MON (N3) 2,696.5	95,386	100 ms	**Met**	N3 still hears HB_N2 directly on B. HB_N1 reaches N3 only through the relay; without it N3 would declare N1 failed

5. Fault-injection test matrix (R4.8 a–f)

How faults are injected. We pull a node's CANH and CANL jumpers for one bus. Pulling N1's or N3's jumper also removes that end's P1 terminator (Section 1).

What the zones do (`common/zephyr/zone_core.c`, `pi_cockpit/cockpit.c`):

- **DEG_x LED:** a node turns on DEG_x when it hasn't heard any other zone's heartbeat on bus x for 60 ms. Relayed copies count. HB_MON checks every 10 ms.
- **System ERROR:** a zone enters ERROR when another zone is lost on **both** buses for 60 ms, or when its CTRL is older than 100 ms. ERROR means hazards on, N3 braked, and N2 still tracking the wheel.
- **Isolated node:** a node cut off from both buses loses everyone, enters its own fail-safe state, and lights both DEG LEDs.
- **Relaying:** N2 and N3 always relay (4.2); the Pi never does.

Restoration (f) is a column on every row. After the jumpers go back in, the DEG LEDs clear within 100 ms, and every zone is back to NORMAL within 500 ms with no reset. A controller that went bus-off recovers on its own (the MCP2515 automatically, the Pi with `restart-ms 100`).

Measured column:

- ✅**:** run on hardware. Status comes from the zone consoles, the Pi status line and the LEDs. Times are scope maxima, with the screenshot linked.
- **"Same path as row N":** not run on its own; the firmware takes the same code path as a row that was measured.

#	Fault (jumpers pulled)	Expected: N1 (Pi)	Expected: N2	Expected: N3	Expected DEG LEDs	Relay needed?	Measured	Restoration (f)
1	N1 off A	NORMAL	NORMAL	NORMAL	N1 DEG_A	No	✅ 10/21: Pi DEG_A on, all NORMAL	✅ DEG off, NORMAL, no reset
2	N1 off B	NORMAL	NORMAL	NORMAL	N1 DEG_B	No	Same path as row 1; N1 off B also ran inside rows 11 and 13 ✅	Same path as row 1
3	N2 off A	NORMAL	NORMAL	NORMAL	N2 DEG_A	No	✅ DEG_A on 62.8 ms, **max 68.6 ms** over 5 (photo 10)	✅ DEG_A off **max 14.8 ms** over 5 (photo 11)
4	N2 off B	NORMAL	NORMAL	NORMAL	N2 DEG_B	No	Same path as row 3; N2 off B ran inside row 10 ✅	Same path as row 3
5	N3 off A	NORMAL	NORMAL	NORMAL	N3 DEG_A	No	Same path as row 3; N3 off A ran inside row 13 ✅	Same path as row 3
6	N3 off B	NORMAL	NORMAL	NORMAL	N3 DEG_B	No	Same path as row 3; N3 off B ran inside row 12 ✅	Same path as row 3
7	N1 off A and B	ERROR (hears no one)	ERROR: N1 lost, CTRL stale	ERROR: N1 lost, CTRL stale; braked	N1 DEG_A + DEG_B	No relay can help	Same path as row 8	Same path as row 8
8	N2 off A and B	NORMAL → ERROR: N2 lost	Fail-safe: ERROR, steering held	ERROR: N2 lost; braked	N2 DEG_A + DEG_B	No relay can help	✅ N3 ERROR **max 68.4 ms** (photo 14); N2 ERROR **max 53.6 ms** (photo 15); limit 100 ms	✅ N2 NORMAL **max 15.6 ms**, N3 **max 16.5 ms** (photos 16–17); limit 500 ms, no reset
9	N3 off A and B	ERROR: N3 lost	ERROR: N3 lost	Fail-safe: ERROR, braked	N3 DEG_A + DEG_B	No relay can help	Same path as row 8	Same path as row 8

#	Fault (jumpers pulled)	Expected: N1 (Pi)	Expected: N2	Expected: N3	Expected DEG LEDs	Relay needed?	Measured	Restoration (f)
10	N1 off A, N2 off B	NORMAL	NORMAL (N1 via N3's relay)	NORMAL, relaying	N1 DEG_A, N2 DEG_B	Yes, **N3**	✅ 10/23: all NORMAL, N3 relaying, CTRL gaps ≤ 28 ms	✅ all NORMAL, no reset
11	N1 off B, N2 off A	NORMAL	NORMAL (N1 via N3's relay)	NORMAL, relaying	N1 DEG_B, N2 DEG_A	Yes, **N3**	✅ 10/23: all NORMAL, N3 relaying, CTRL gaps ≤ 33 ms	✅ all NORMAL, no reset
12	N1 off A, N3 off B	NORMAL	NORMAL, relaying	NORMAL (N1 via N2's relay)	N1 DEG_A, N3 DEG_B	Yes, **N2**	✅ 10/23: all NORMAL, N2 relaying, CTRL gaps ≤ 41 ms	✅ all NORMAL, no reset
13	N1 off B, N3 off A (analyzed case)	NORMAL	NORMAL, relaying	NORMAL (N1 via N2's relay)	N1 DEG_B, N3 DEG_A	Yes, **N2**	✅ 10/23: all NORMAL. Relayed brake **1.1 ms** (photo 3); RELAY C **405 µs** (M6)	✅ all NORMAL, no reset
14	N2 off A, N3 off B	NORMAL (hears N2 on B, N3 on A)	ERROR: N3 lost	ERROR: N2 lost; braked	N2 DEG_A, N3 DEG_B	Yes, via **N1**: not supported	Not tested; the TA clarification excludes it (it would need Linux to relay). Fails safe: N3 brakes, hazards on	Same path as row 8
15	N2 off B, N3 off A	NORMAL (hears N2 on A, N3 on B)	ERROR: N3 lost	ERROR: N2 lost; braked	N2 DEG_B, N3 DEG_A	Yes, via **N1**: not supported	Not tested; excluded, as row 14. Fails safe	Same path as row 8
16	All nodes off A (e)	NORMAL on B	NORMAL on B	NORMAL on B	All three DEG_A	No	✅ 10/21: all DEG_A, all NORMAL, CTRL gaps 36–39 ms	✅ all back on both buses, no reset
17	All nodes off B (e)	NORMAL on A	NORMAL on A	NORMAL on A	All three DEG_B	No	Same path as row 16	Same path as row 16

Two nodes off the same bus (N1+N2, N1+N3 or N2+N3 off A, or off B) behave exactly like rows 16–17. The one node still on that bus hears no other zone, so all three show DEG on that bus, and everything runs NORMAL on the other bus.

6. Team-defined values and Lab 1 revision

`Doc/lab1/Requirement_Document.md` has been updated for Lab 3:

- **Section 6:** the template catalog, with our payloads, handling rules and utilization.
- **Section 7:** a "Lab 3 values" table with every threshold and mapping and its source file.
- **Revision history:** a 2026-10-21 entry, including the relay limitation.

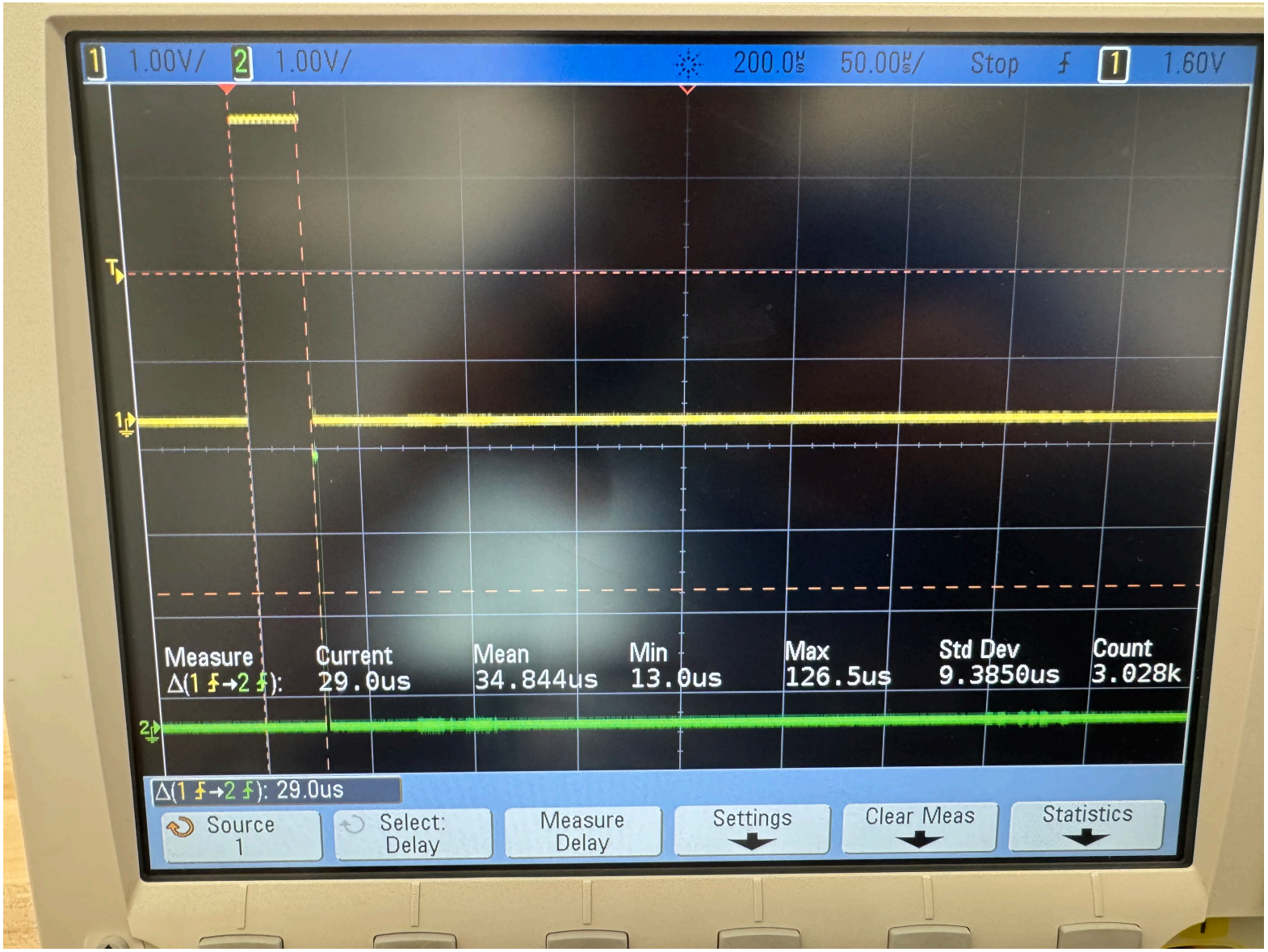
Appendix A: Scope captures

Every value is the **maximum over the repeats** listed. Test-point rows refer to Figure 1.

A.1 Required measurements (TA clarification, M1–M6)

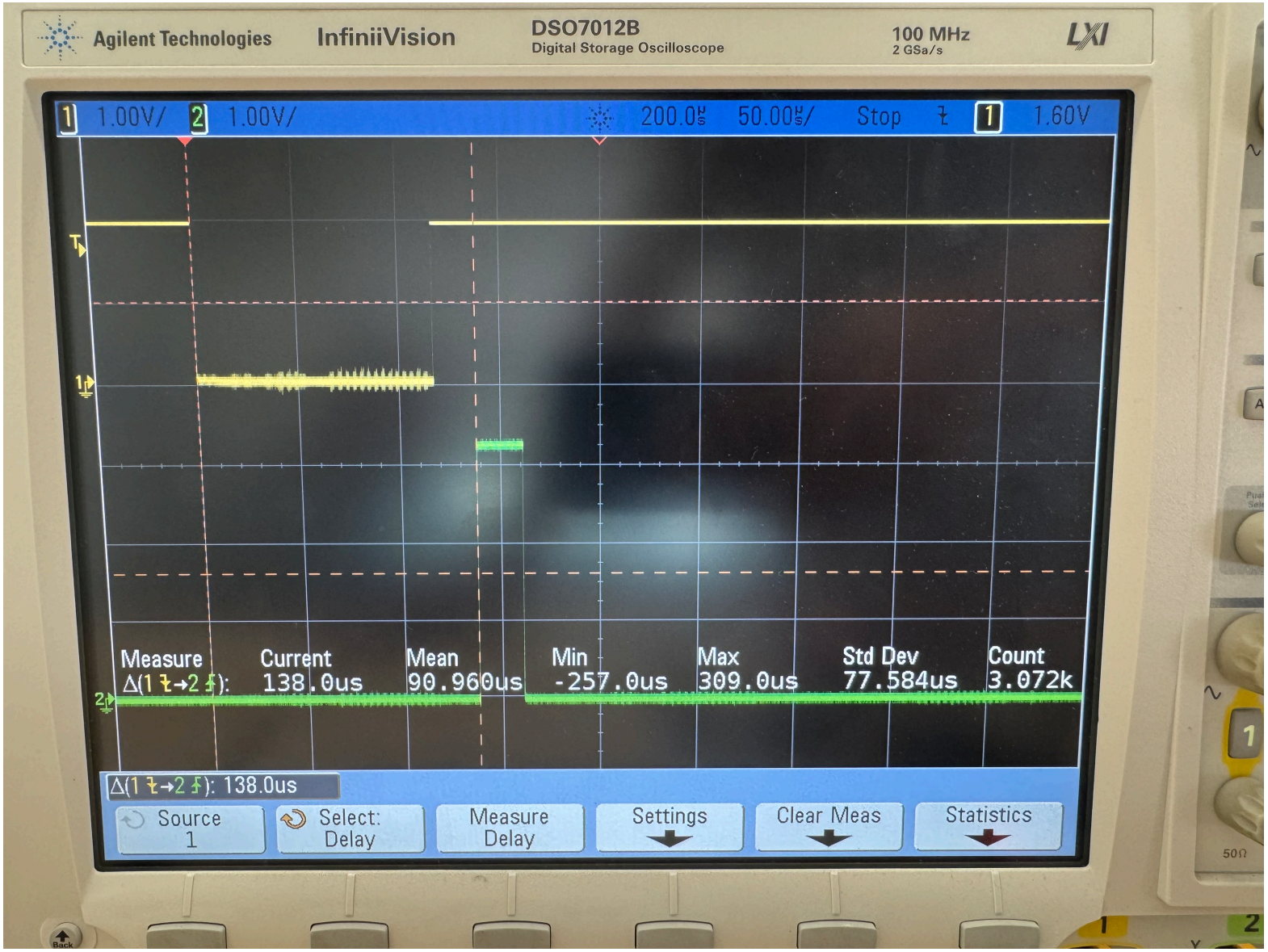
The six measurements the RTA in Sections 3–4 uses. Each screenshot shows both probes and the max.

M1 — Measurement #1: Pi stage, UDP_RX → CMD_TX (cockpit `--rt`, normal load). Max 126.5 µs, mean 34.9 µs, 3000 samples



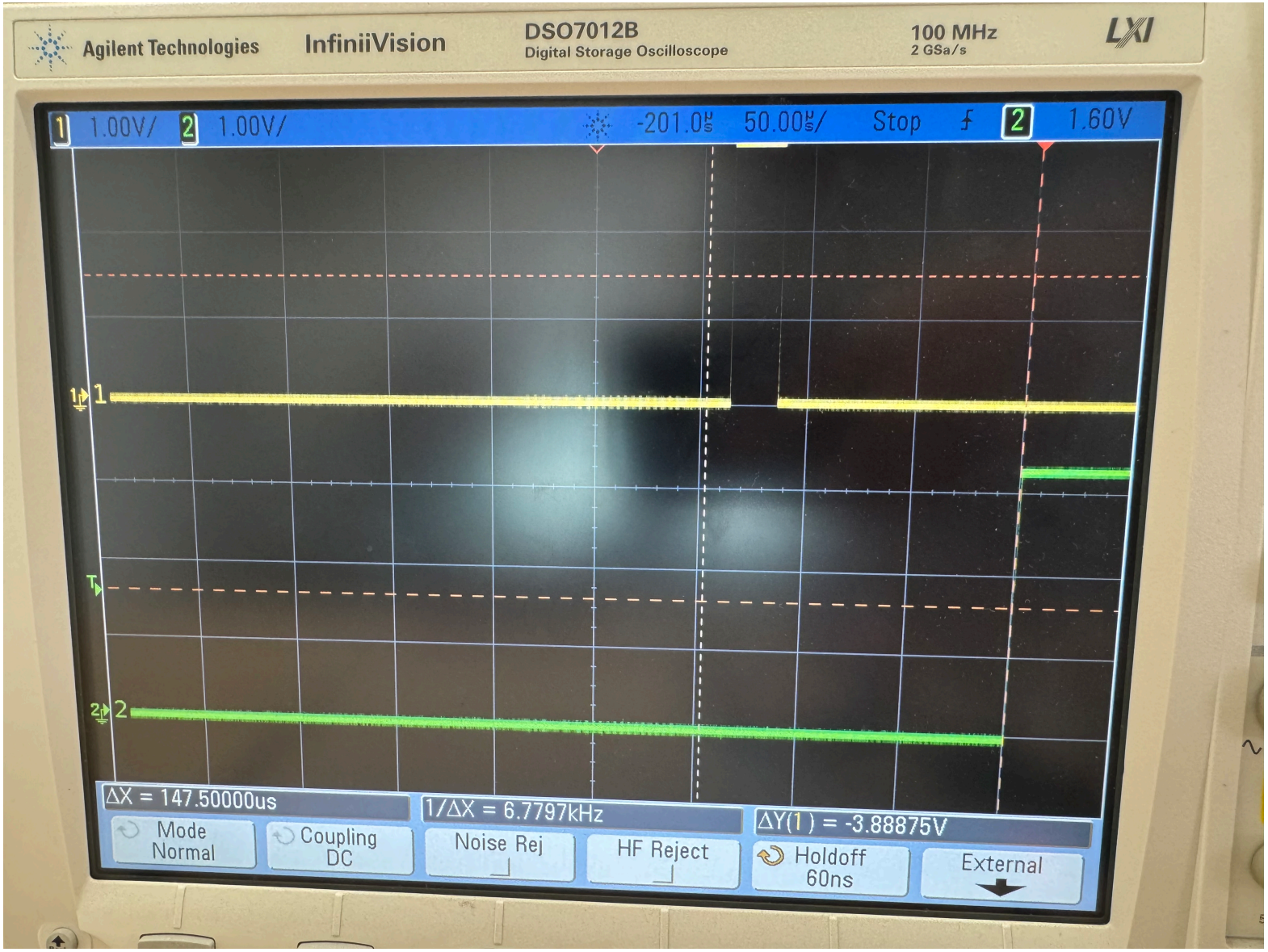
M1: Pi stage UDP_RX → CMD_TX, cockpit --rt, normal load. Max 126.5 µs

M2 — Measurement #2: CAN_RX C, N3 (CAN A INT falling → CAN_RX_A). Max 309 µs



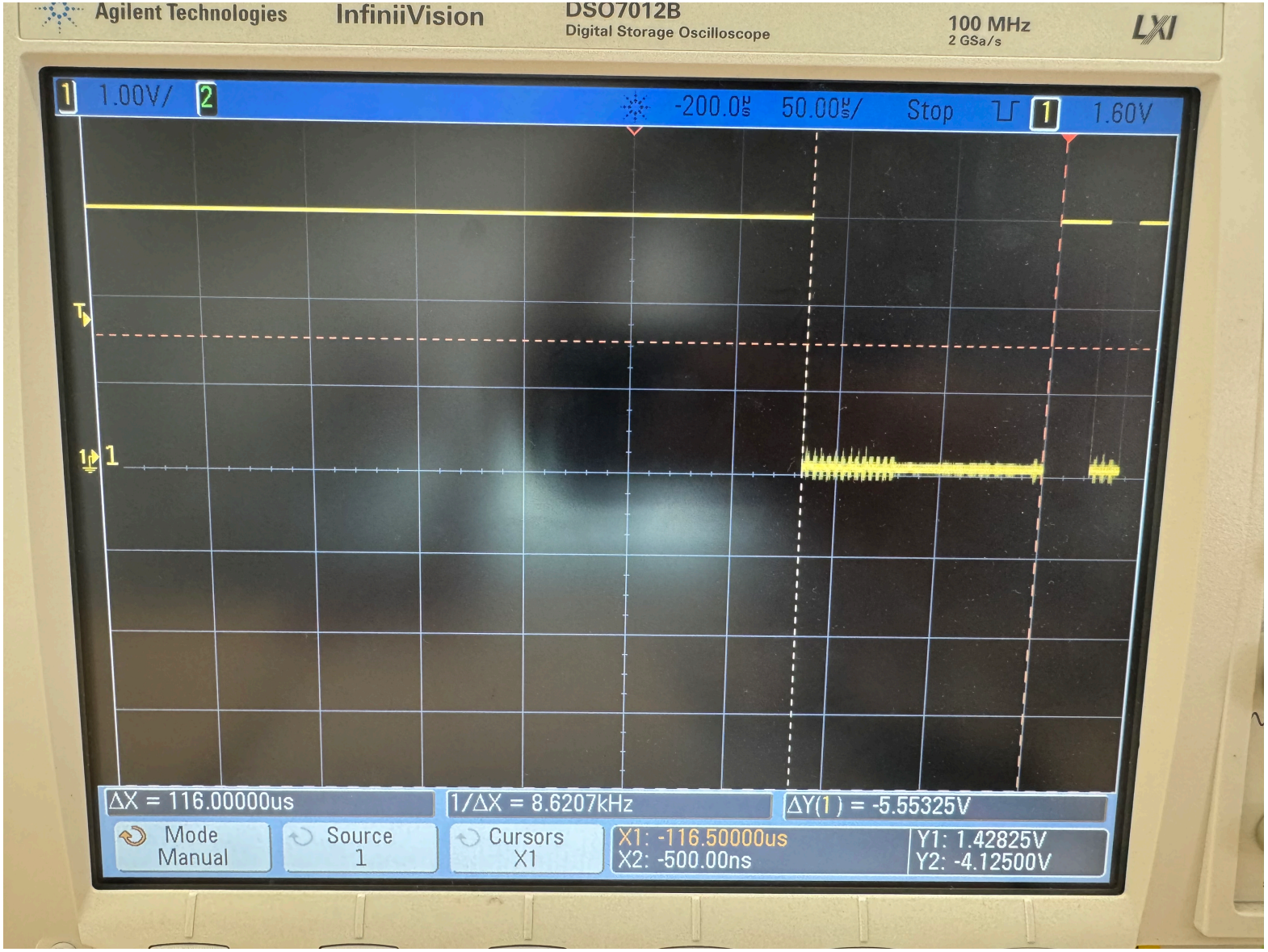
M2: Measurement #2, CAN_RX C, N3

M3 — Measurement #3: DRIVE C, N3 (CAN_RX_A → DIR_A on a brake frame). Max 147 µs



M3: Measurement #3, DRIVE C, N3

M4 — Measurement #4: SPI frame load (B) (CS low → high, longest window). Max 116.5 µs

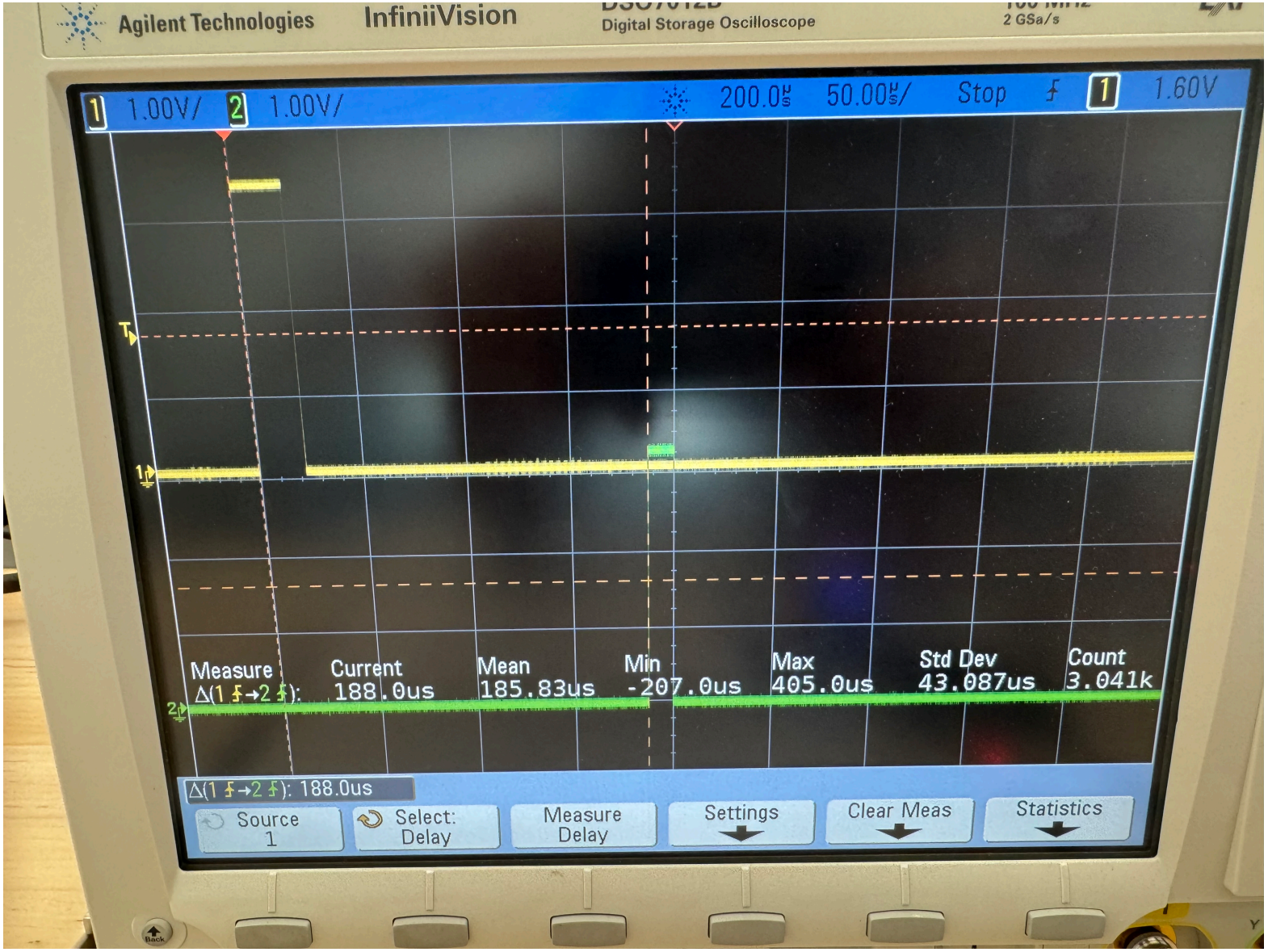


M4: Measurement #4, SPI frame load (B)

M5 — Measurement #5: CAN_RX C, N2 (CAN A INT falling → CAN_RX_A). Max 289.5 µs

M5: Measurement #5, CAN_RX C, N2

M6 — Measurement #6: RELAY C, N2 (CAN_RX_A → RELAY_TX, N1 off B and N3 off A). Max 405 µs



M6: Measurement #6, RELAY C, N2

A.2 Additional captures (not required)

End-to-end chains and fault-injection timings taken before the clarification. They back the fault matrix (Section 5) and Appendix B. Photos 1–5 were taken with the cockpit **without** `--rt`, so their Pi stage is the slower one (up to about 443 µs) and the chain values are conservative. Photo 6 became M1 above.

Brake chain (R2.2), Pi UDP_RX → N3 DIR_A. One capture 640 µs; max 940 µs over 15 counts

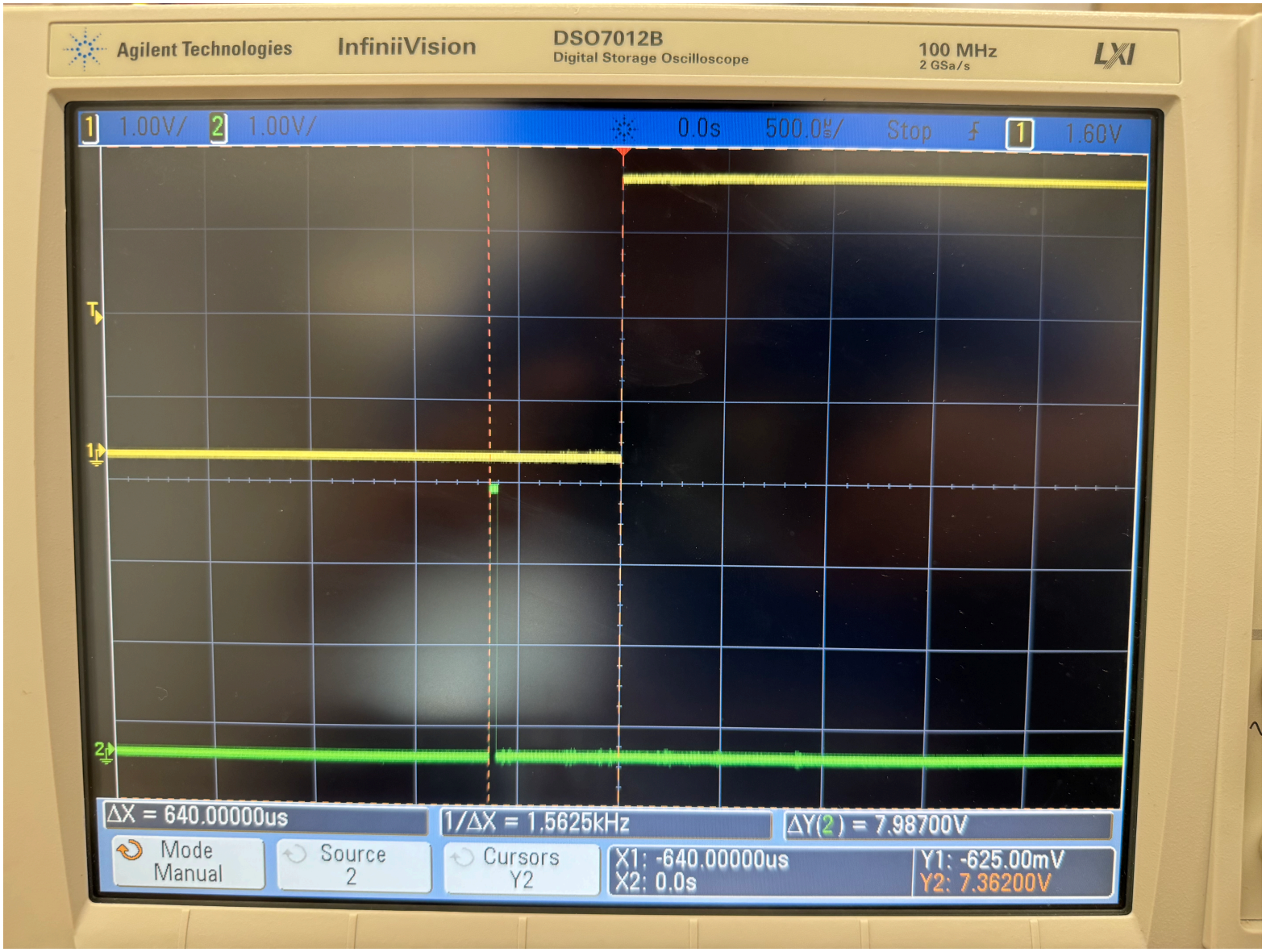


Photo 1: Brake chain (R2.2), Pi UDP_RX → N3 DIR_A. One capture 640 µs; max 940 µs over 15

Throttle chain (R2.1), Pi UDP_RX → N3 PWM_SET (first pulse). One capture 614 µs; max 626 µs over 10 counts

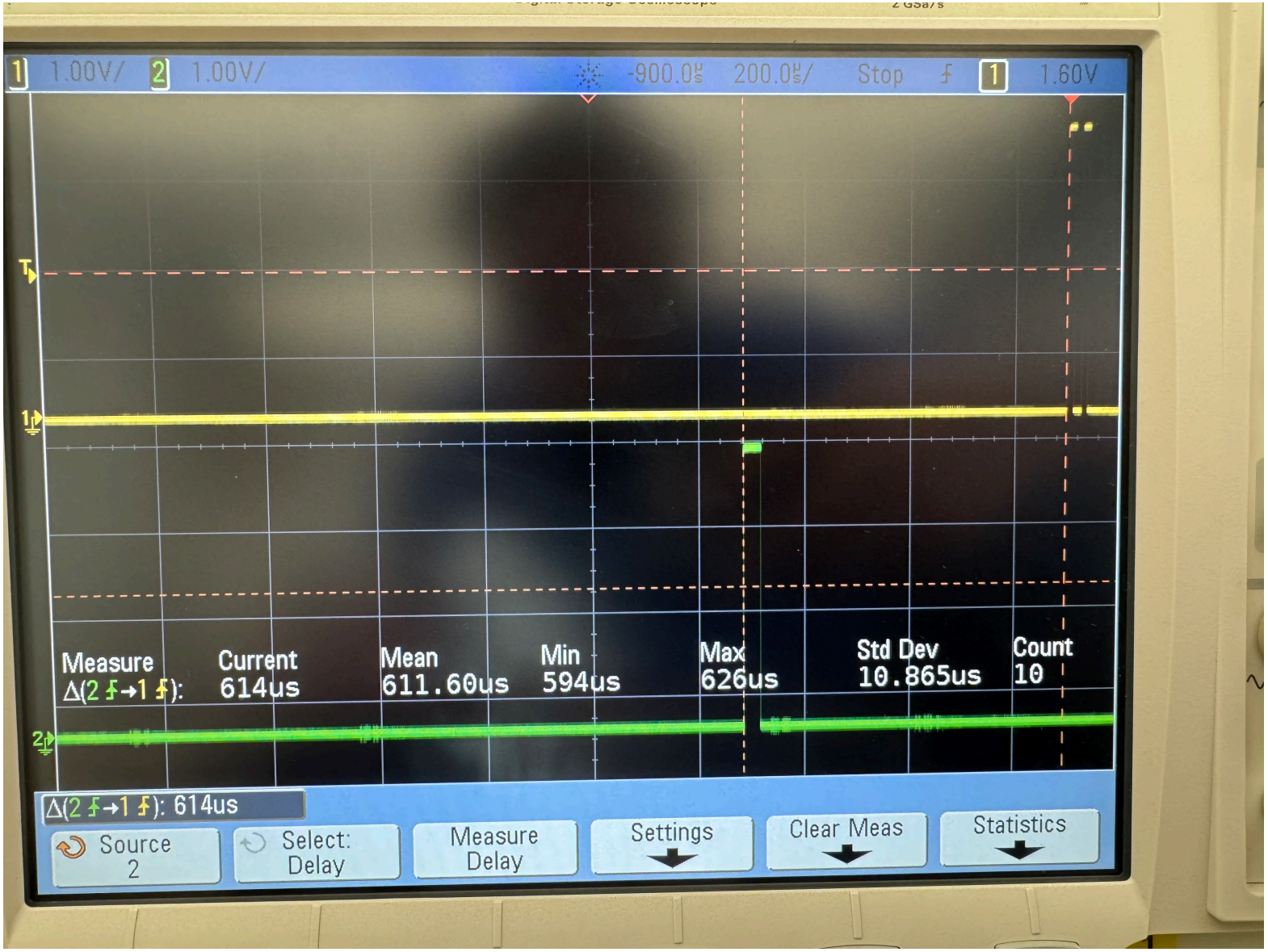


Photo 2: Throttle chain (R2.1), Pi UDP_RX → N3 PWM_SET (first pulse). One capture 614 µs; max 626 µs over 10

Relayed brake chain, scenario (c), Pi UDP_RX → N3 DIR_A via N2 RELAY. Max 1.1 ms over 10 counts

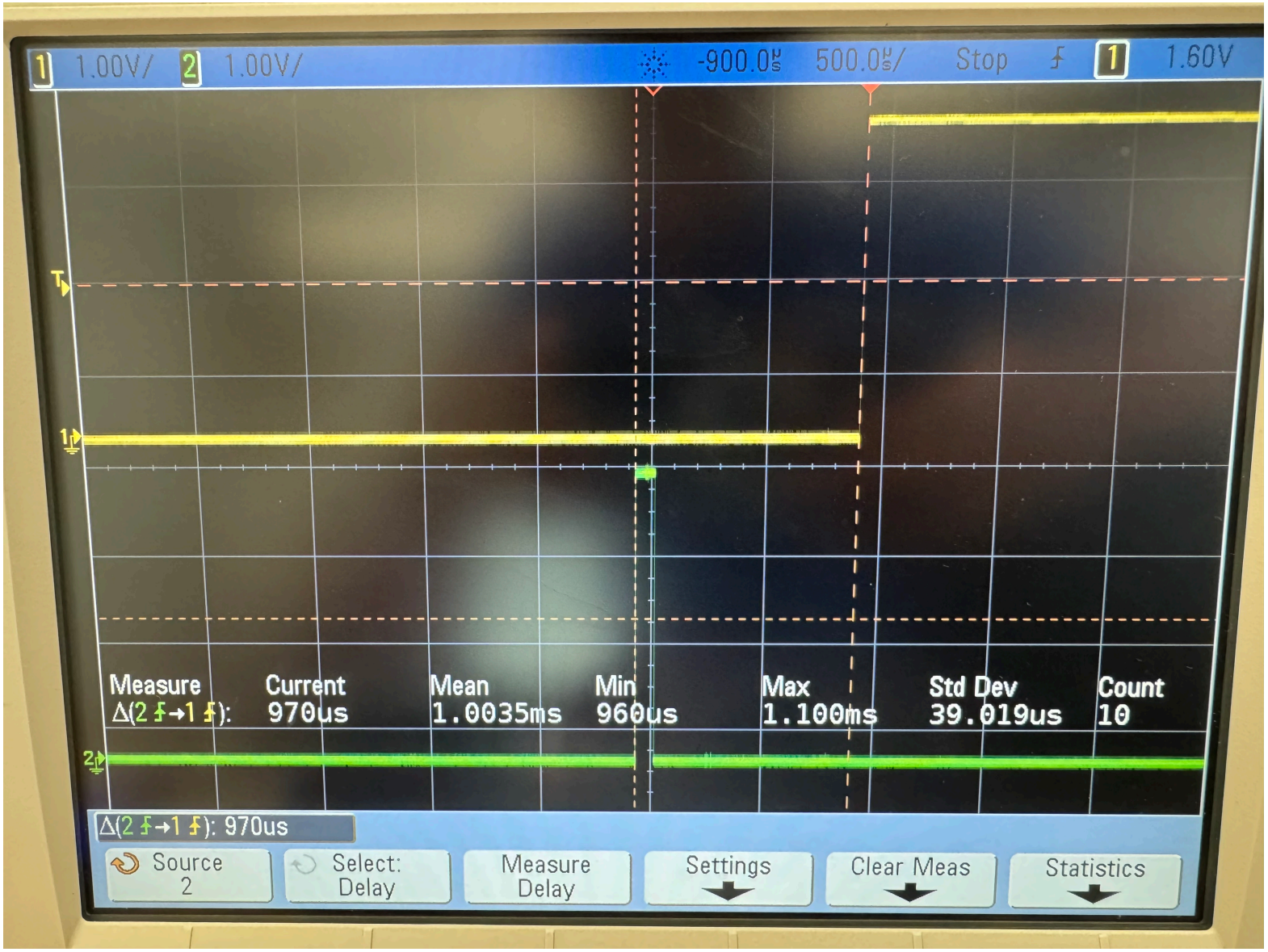


Photo 3: Relayed brake chain, scenario (c), Pi UDP_RX → N3 DIR_A via N2 RELAY. Max 1.1 ms over 10

Relay stage, scenario (c), Pi UDP_RX → N2 RELAY_TX. One capture 474 µs; max 607 µs over 10 counts

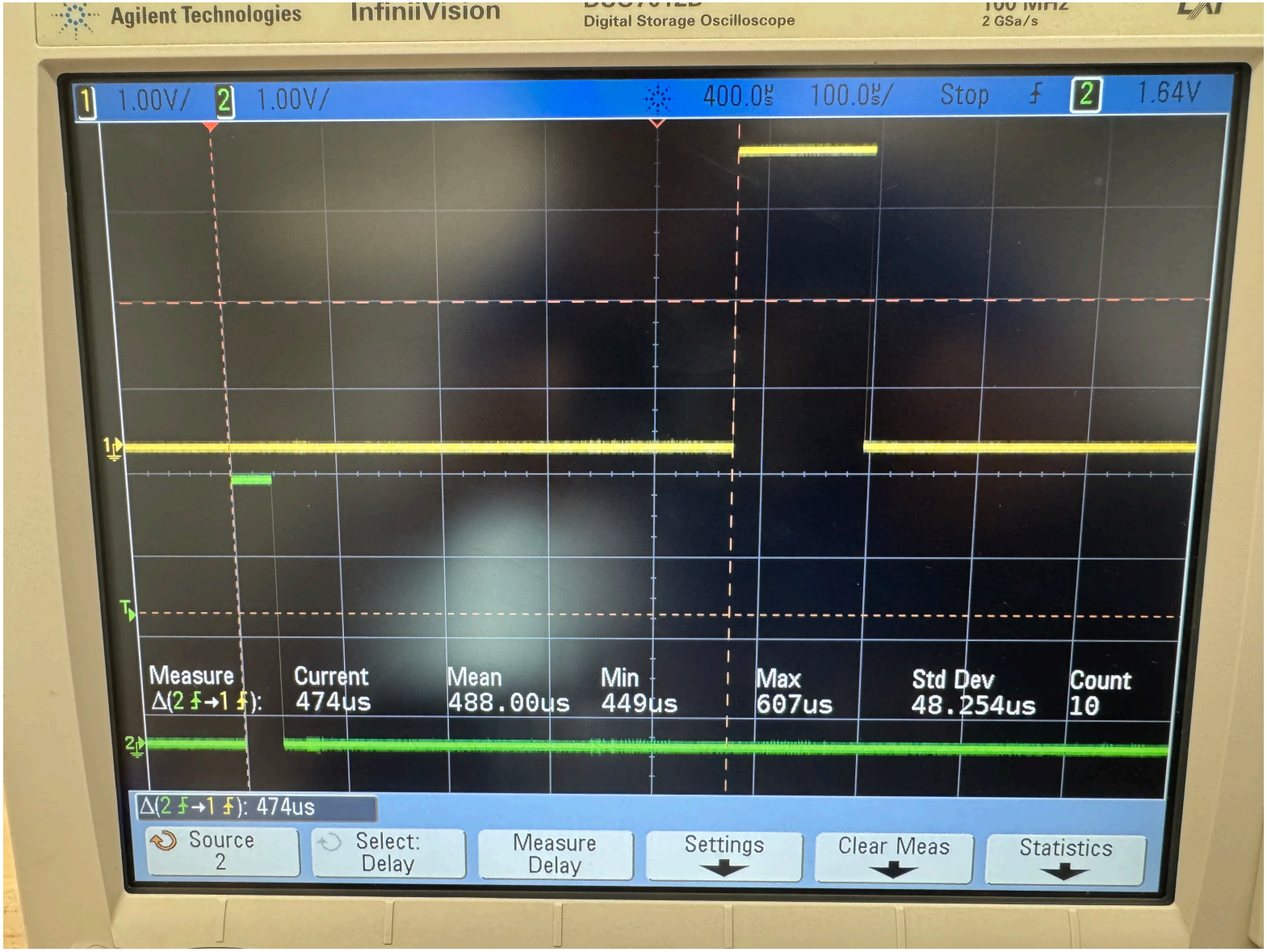


Photo 4: Relay stage, scenario (c), Pi UDP_RX → N2 RELAY_TX. One capture 474 µs; max 607 µs over 10

Blinker sync (R2.4), N2 FL vs N3 RL rising edges. One capture 186 µs; max 768 µs over 10 counts

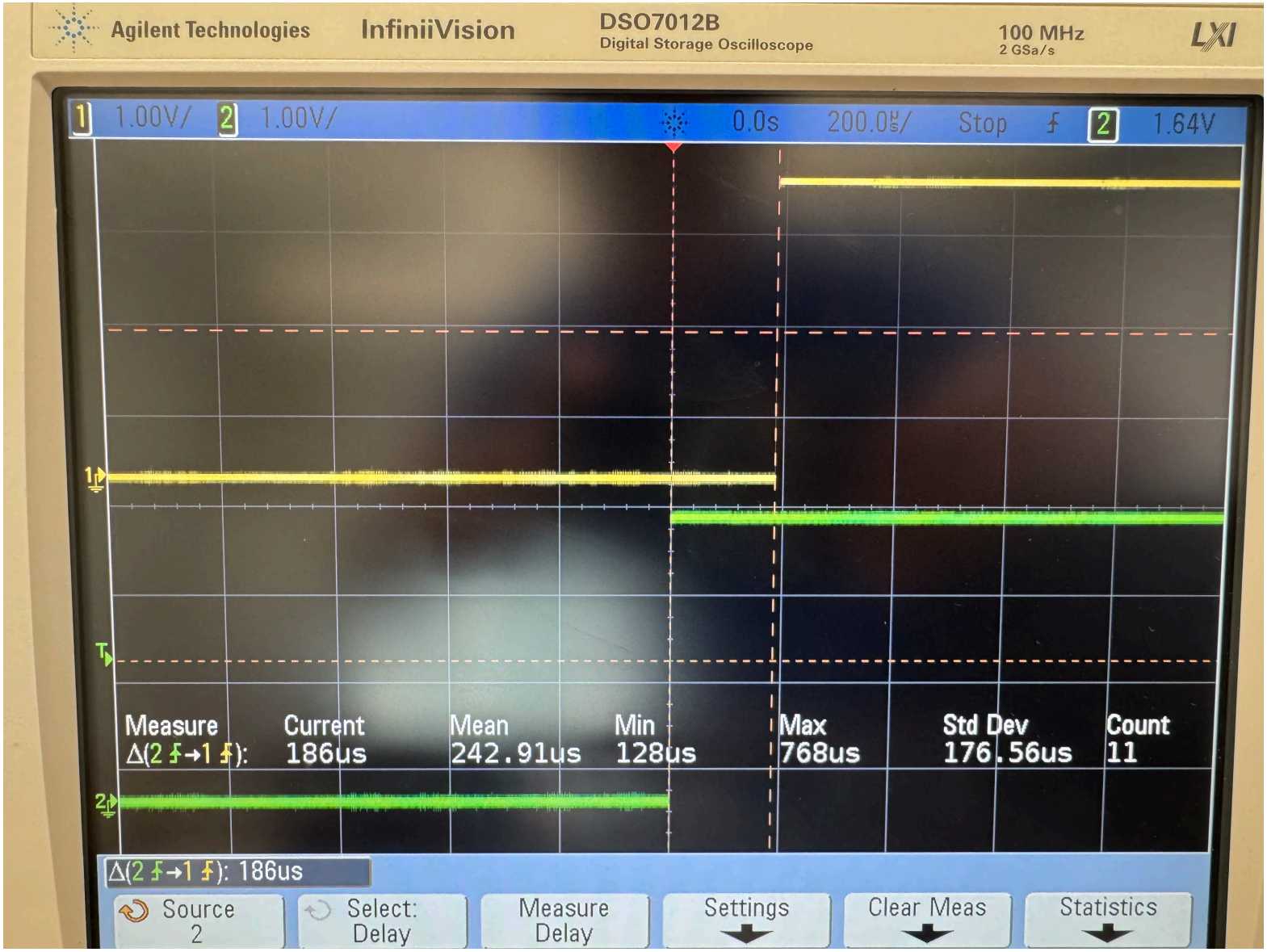


Photo 5: Blinker sync (R2.4), N2 FL vs N3 RL rising edges. One capture 186 µs; max 768 µs over 10 edges

Pi stage, cockpit --rt, under stress --cpu 4 --io 2. Max 82 µs, mean 32.1 µs, 3000 samples

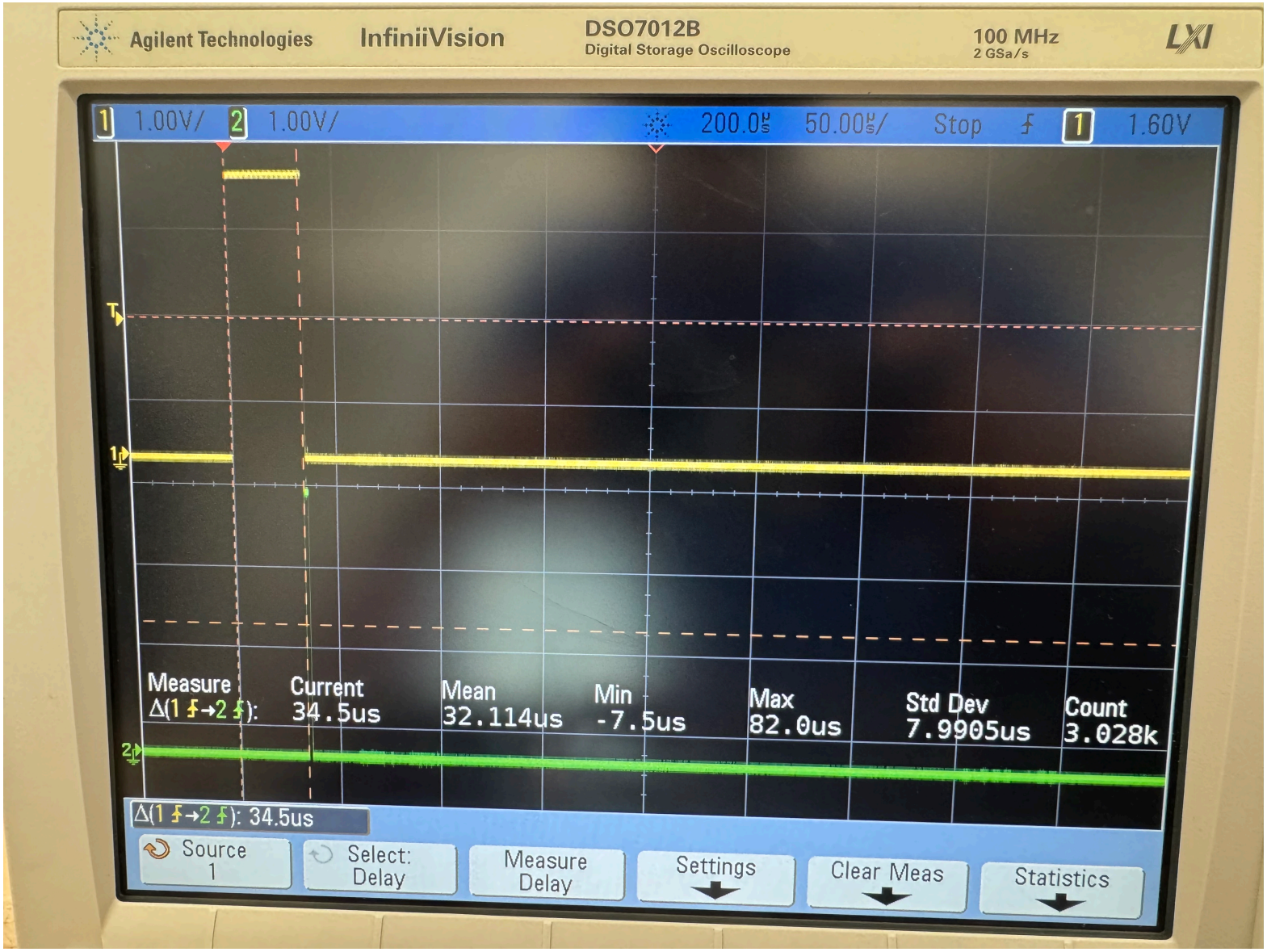


Photo 7: Pi stage, cockpit --rt, under stress --cpu 4 --io 2. Max 82 µs, mean 32.1 µs, 3000 samples

CAN A CANH/CANL. Idle 2.18/2.21 V; dominant 2.68/0.61 V, differential 2.07 V

Photo 8: CAN A CANH/CANL. Idle 2.18/2.21 V; dominant 2.68/0.61 V, differential 2.07 V

CAN B CANH/CANL. Idle 2.20/2.19 V; dominant 2.70/0.73 V, differential 1.97 V

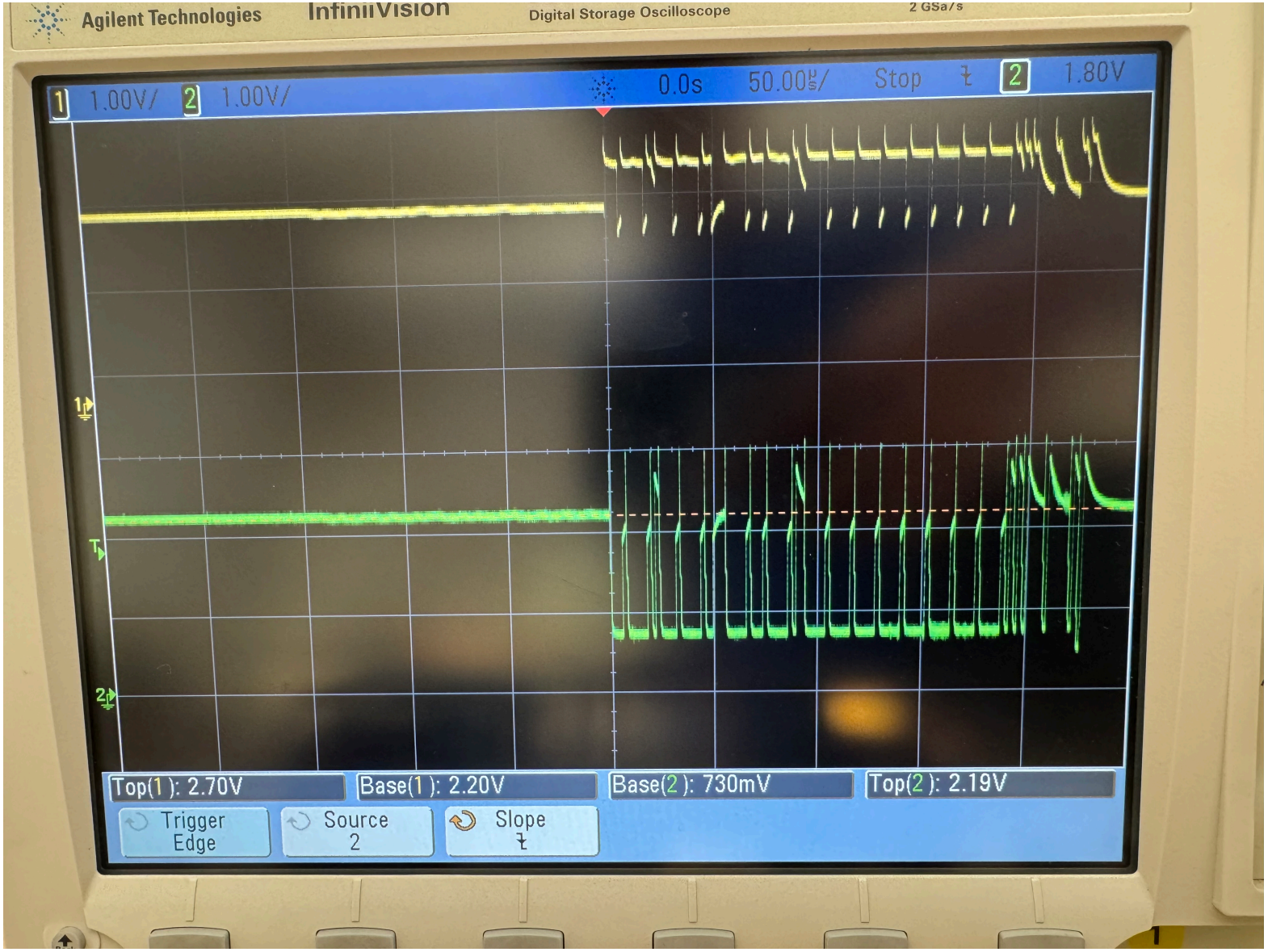


Photo 9: CAN B CANH/CANL. Idle 2.20/2.19 V; dominant 2.70/0.73 V, differential 1.97 V

(a) N2 off CAN A, last CAN_RX_A → DEG_A on. 62.8 ms; max 68.6 ms over 5 counts

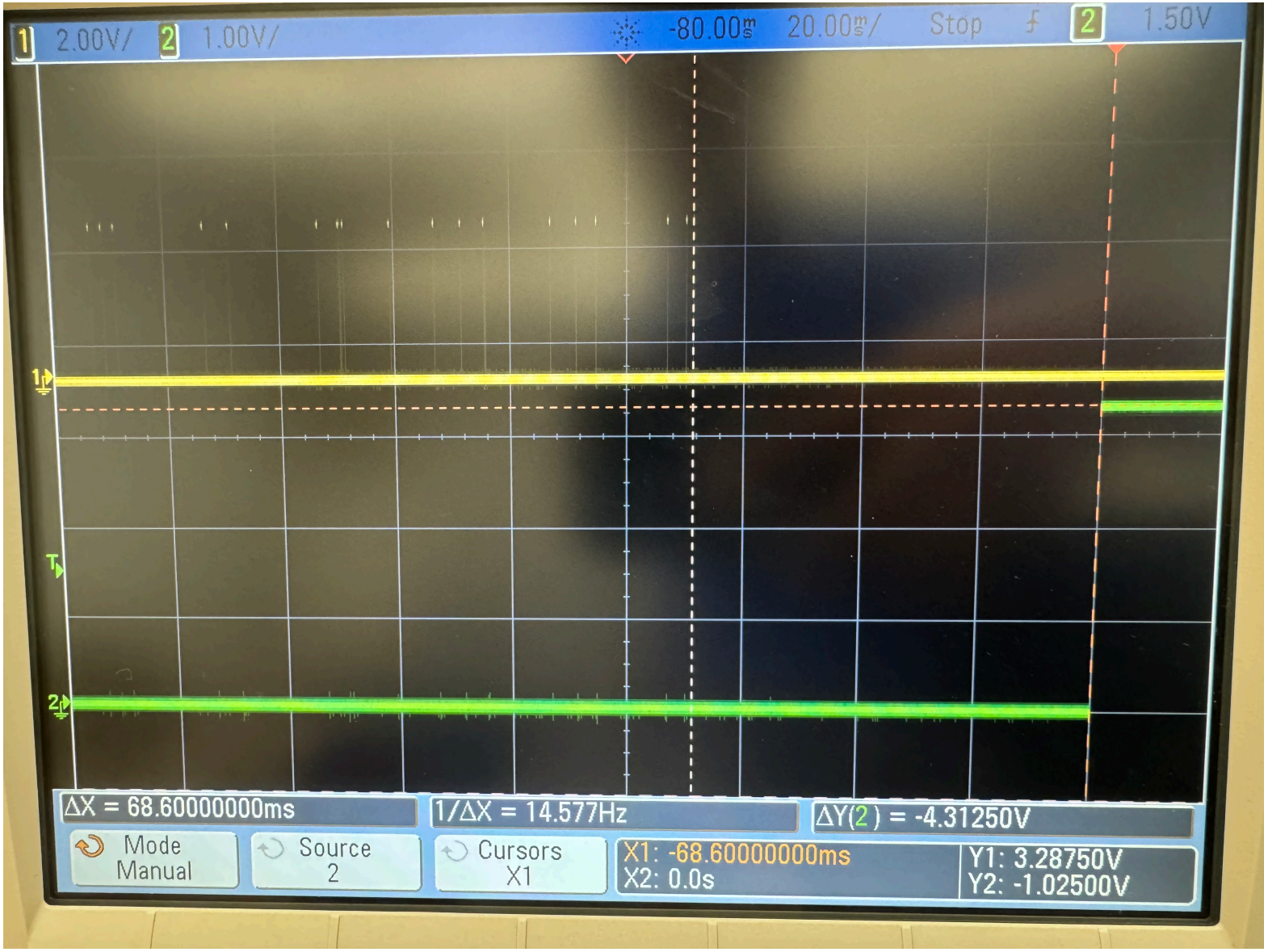


Photo 10: (a) N2 off CAN A, last CAN_RX_A → DEG_A on. 62.8 ms; max 68.6 ms over 5

(f) N2 bus A restored, first CAN_RX_A → DEG_A off. 9.6 ms; max 14.8 ms over 5 counts

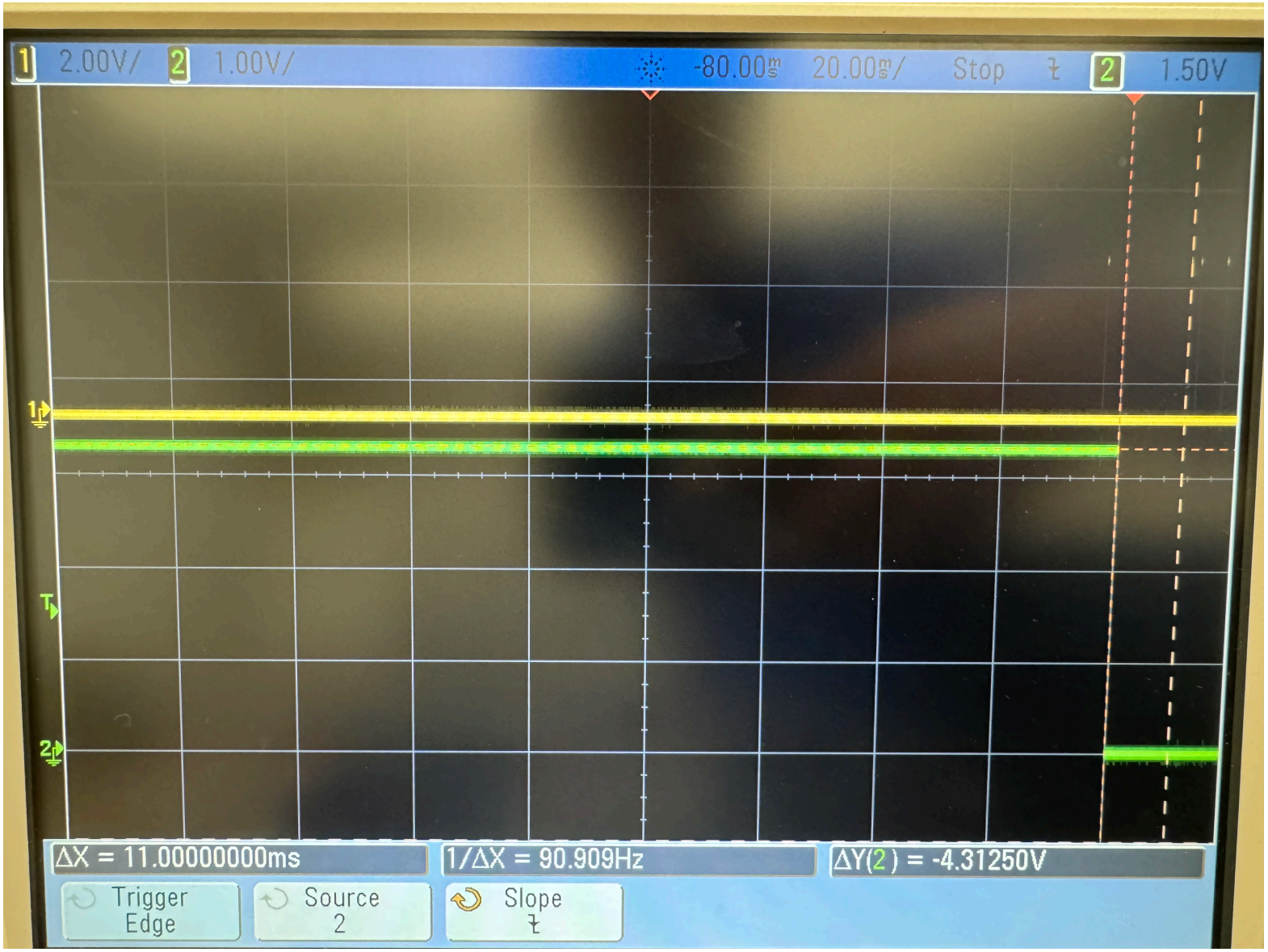


Photo 11: (f) N2 bus A restored, first CAN_RX_A → DEG_A off. 9.6 ms; max 14.8 ms over 5

R2.5 local fail, N2 B1 press → N2 ERR. Max 20 µs over 5 counts

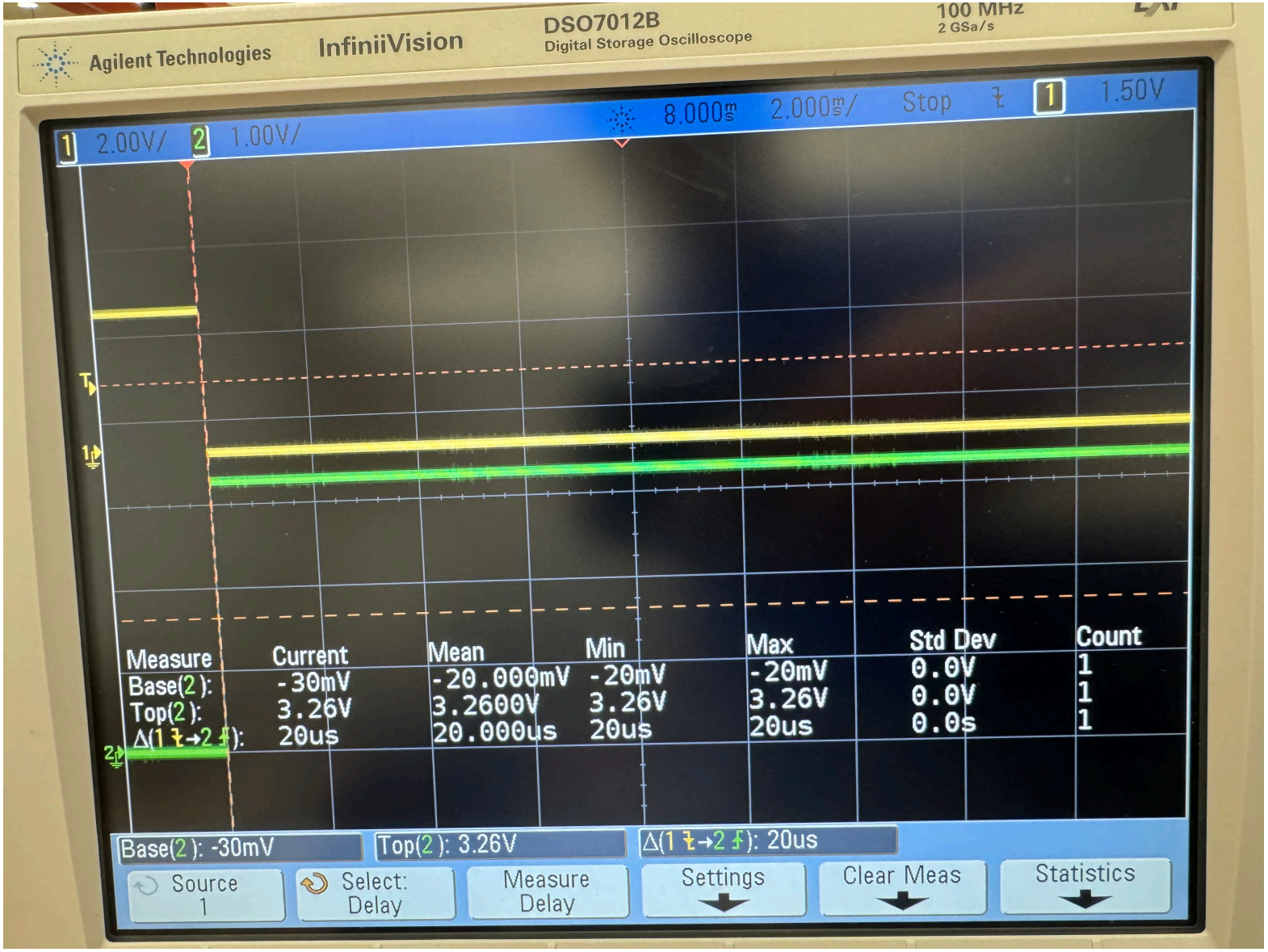


Photo 12: R2.5 local fail, N2 B1 press → N2 ERR. Max 20 µs over 5

R2.5 system error, N2 B1 press → N3 ERR. 1.0 ms; max 1.1 ms over 5 counts

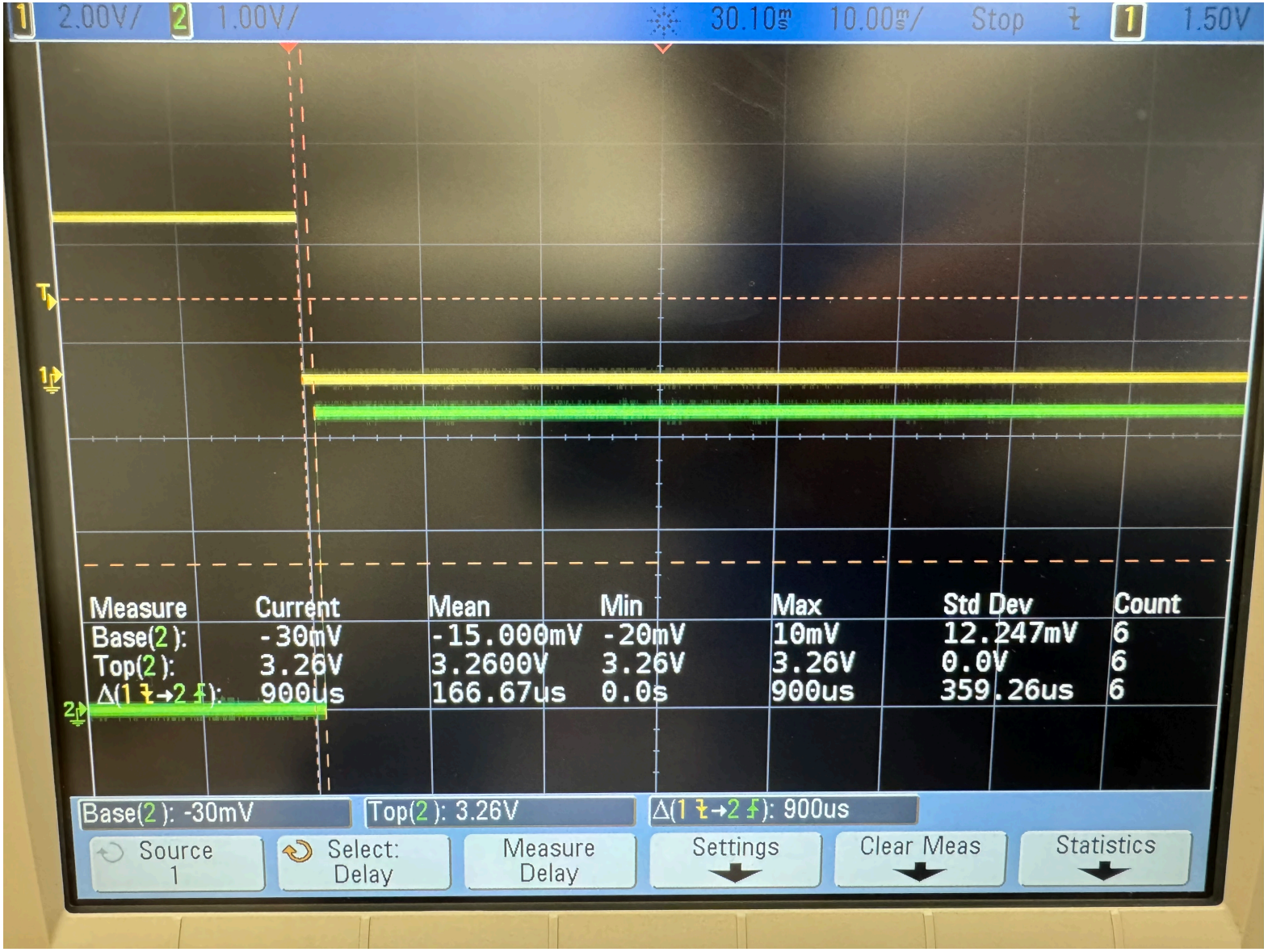


Photo 13: R2.5 system error, N2 B1 press → N3 ERR. 1.0 ms; max 1.1 ms over 5

(d) N2 off both buses, last N2 CAN_RX_B → N3 ERR. 59.4 ms; max 68.4 ms over 3 counts

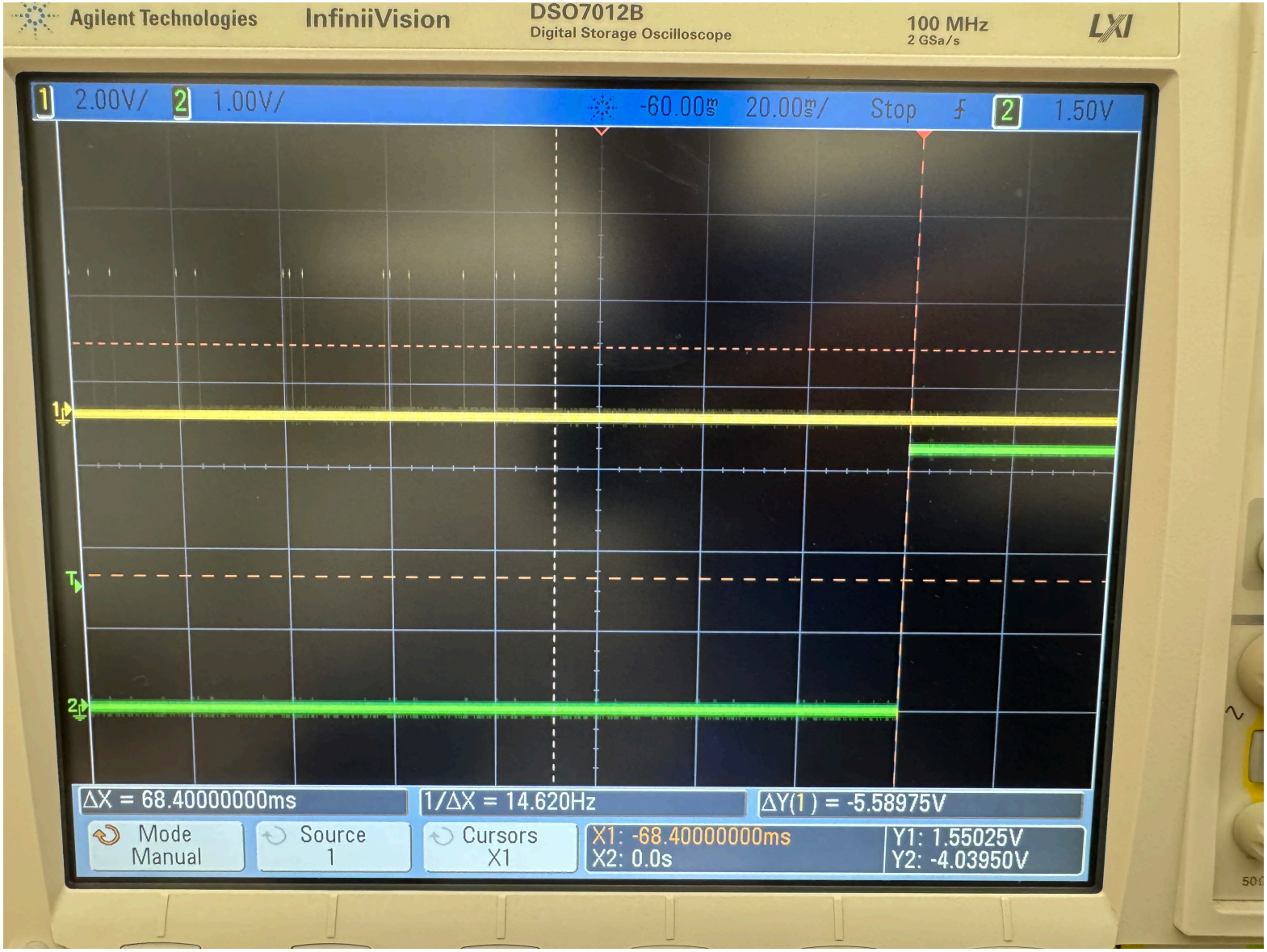


Photo 14: (d) N2 off both buses, last N2 CAN_RX_B → N3 ERR. 59.4 ms; max 68.4 ms over 3

(d) isolated N2 enters fail-safe, last CAN_RX_B → N2 ERR. 52.4 ms; max 53.6 ms over 3 counts

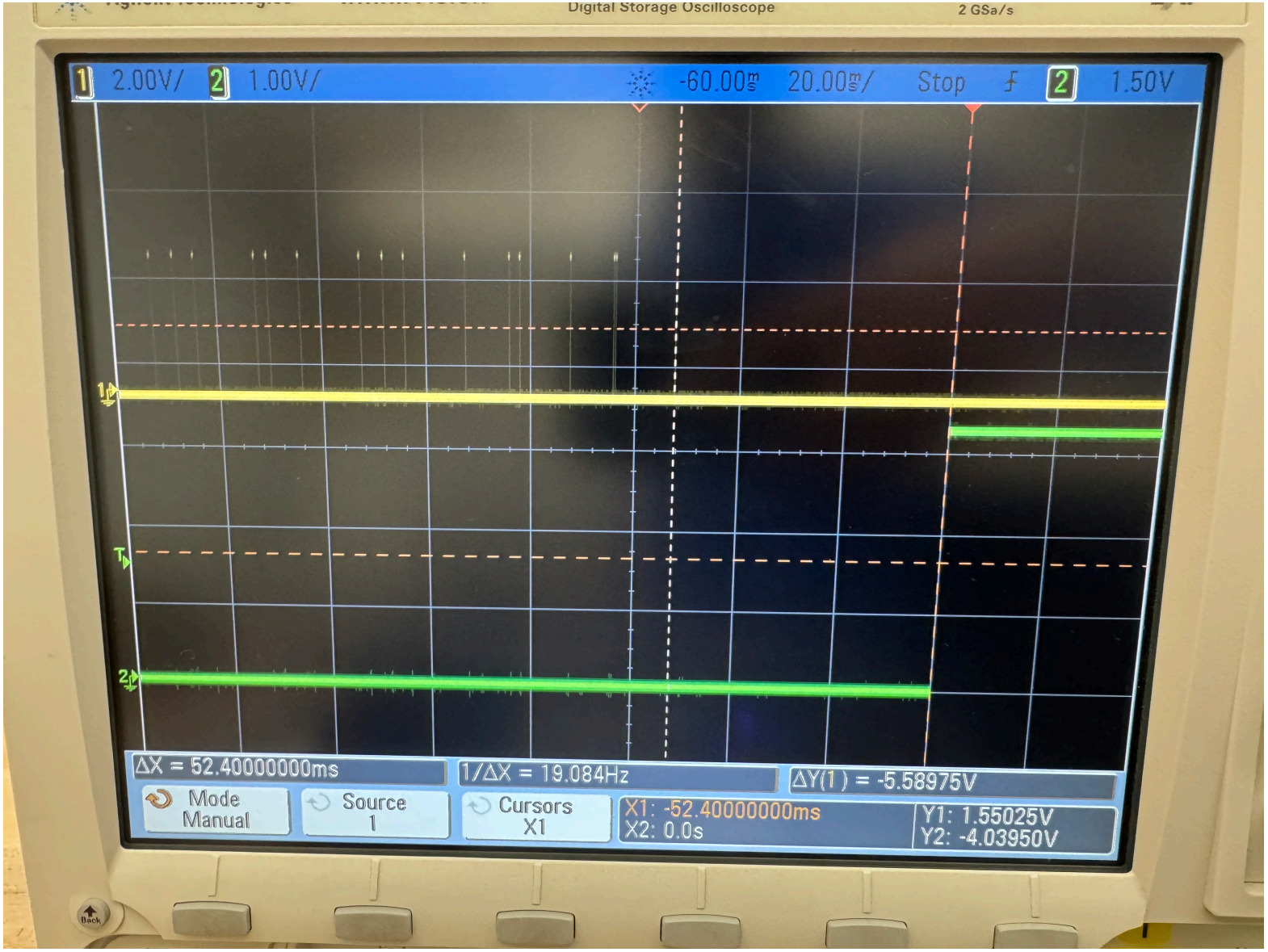


Photo 15: (d) isolated N2 enters fail-safe, last CAN_RX_B → N2 ERR. 52.4 ms; max 53.6 ms over 3

(f) N2 jumpers restored, first CAN_RX_A → N2 ERR off. 3.6 ms; max 15.6 ms over 3 counts

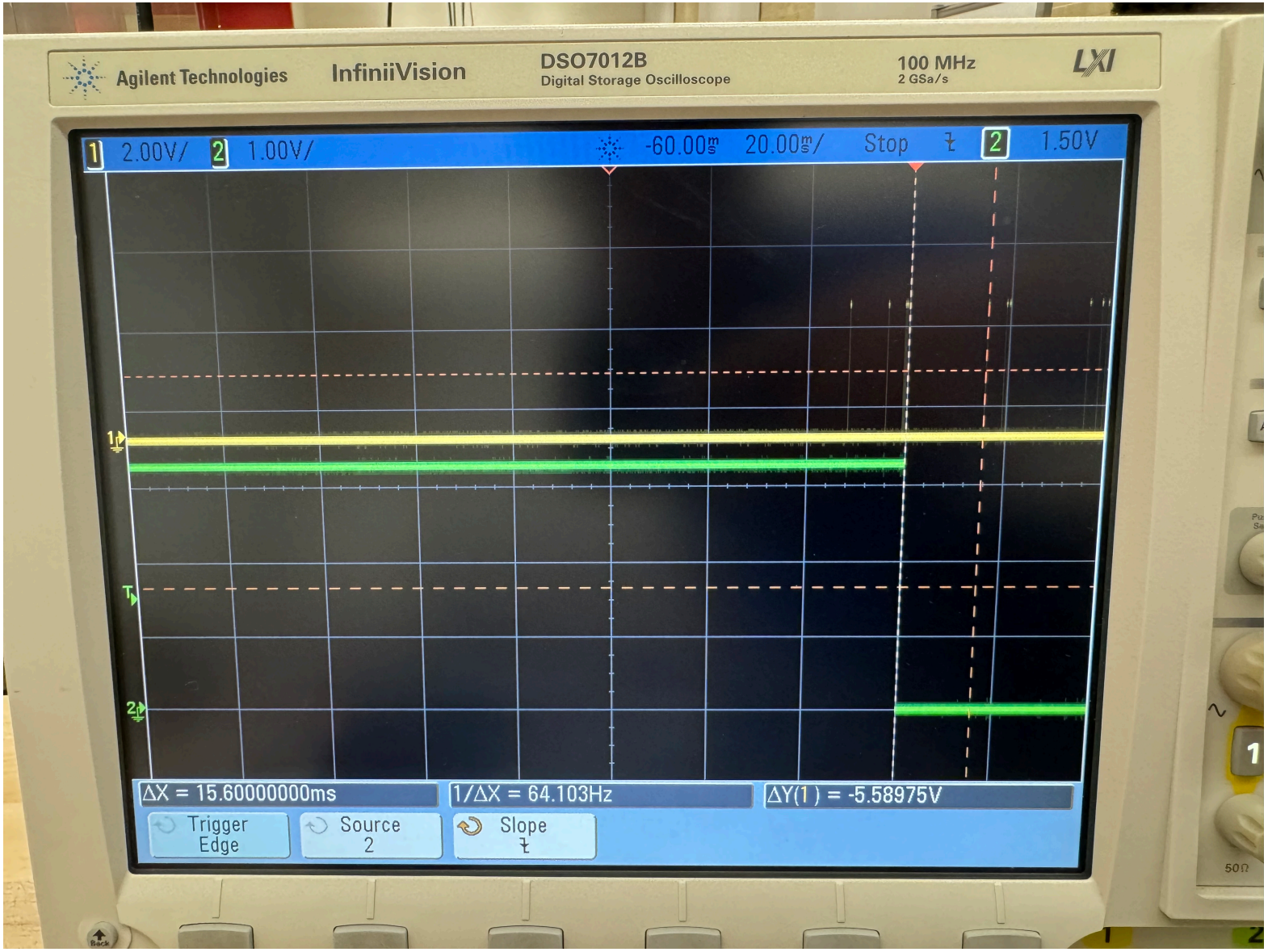


Photo 16: (f) N2 jumpers restored, first CAN_RX_A → N2 ERR off. 3.6 ms; max 15.6 ms over 3

(f) N2 restored, first N2 CAN_RX_A → N3 ERR off. Max 16.5 ms over 3 counts

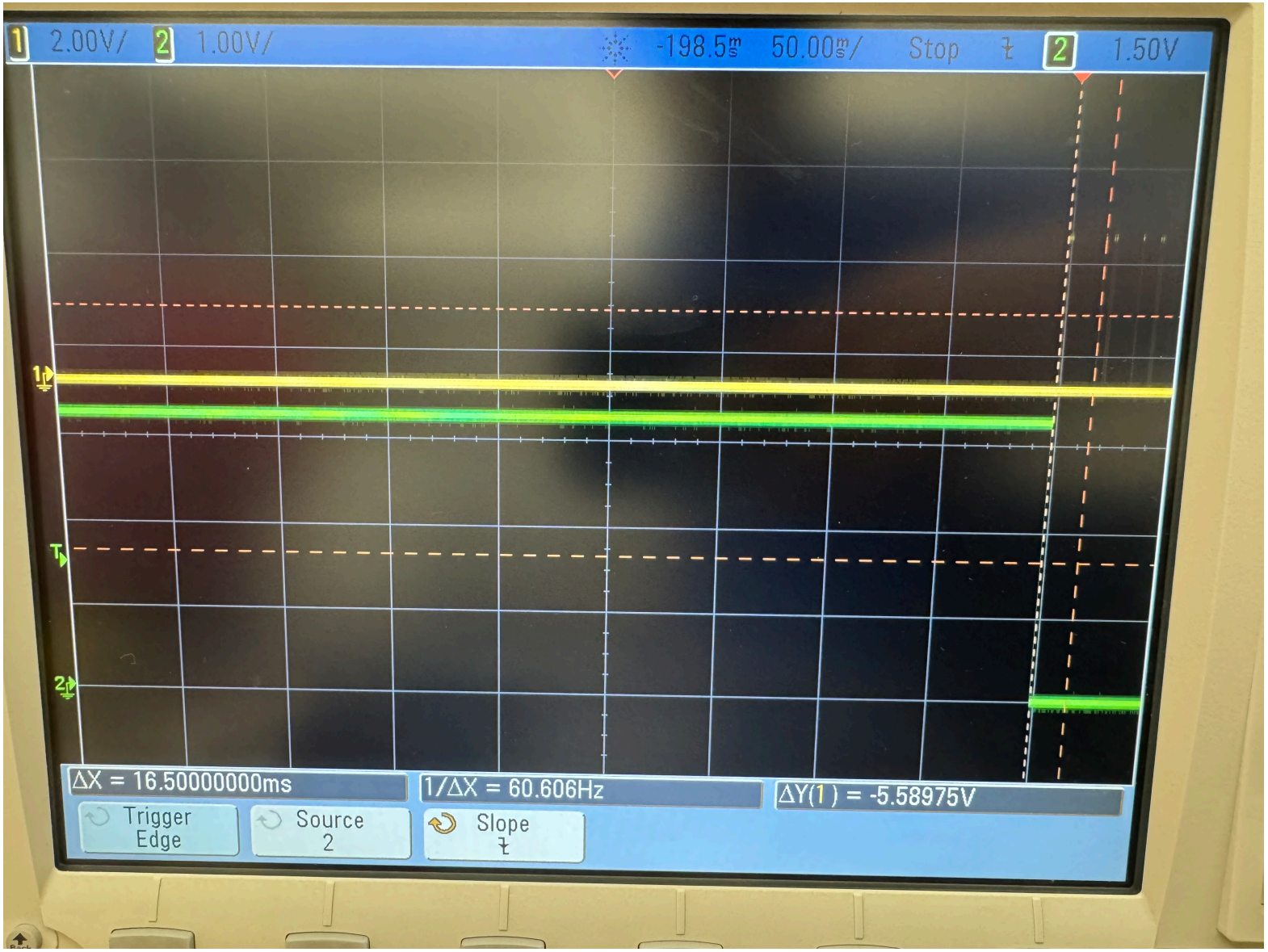


Photo 17: (f) N2 restored, first N2 CAN_RX_A → N3 ERR off. Max 16.5 ms over 3

Appendix B: Analysis with measured WCETs (both buses, relay load)

This is our analysis before the TA clarification. It goes further than the template in Sections 3–4:

- it uses our own measured WCETs instead of the placeholders;
- it charges both buses and every relayed copy at once;
- it models the MCP2515 driver's real per-frame cost.

It explains the gap between the template bounds and the scope results. Inside this appendix, references to "3.x" and "4.x" mean its own subsections B.1–B.5. The relay rows describe the design before N3 also relayed; Section 4 has the current design.

The tables below already include both buses and N2's relayed copies, which is the worst case; on one bus each zone receives fewer frames. A small script iterates every equation to convergence.

B.1 Pi stage (measured)

Method: scope Delay statistics from UDP_RX to CMD_TX, 60 s per run, 3000 samples.

Cockpit scheduling	Load	Max	Mean
`--rt` (SCHED_FIFO 80, pinned to CPU 3)	normal (M1)	**126.5 µs**	34.9 µs
`--rt`	`stress --cpu 4 --io 2` (photo 7)	82 µs	32.1 µs
default	normal / stress	442.5 / 444.5 µs	53.0 / 104.0 µs

- **Value used: J_PI = 126.5 µs**, the larger of the two `--rt` maxima. SCHED_FIFO cut the worst case about 3.5×, so the cockpit always runs with `--rt`.
- **No 99th percentile:** the scope reports max and mean only.
- **Not covered:** the laptop → Pi network delay, which comes before UDP_RX. The demo uses Ethernet; over Wi-Fi we saw gaps of up to 139 ms.

B.2 Node response-time analysis

How C was measured.

- **Method:** each job reads its own thread's execution cycles (`k_thread_runtime_stats_get`, 84 MHz; `common/zephyr/wcet.c`) at start and end. That excludes preemption and includes any ISR that hit the job, which is conservative.
- **Value used:** the maximum over every run (driving, blinkers, test button, bad input, jumper pulls), printed by each zone. Job counts are in the progress doc.
- **Per-frame receive cost:** C_ev = DRV_FRAME + RX_DISPATCH.
  - DRV_FRAME is the MCP2515 thread's run time between two of our dispatches on the same bus: its SPI reads plus any TX-done handling.
  - It is taken from a clean 5-minute run with no jumper pulls. While a node is unplugged, seconds of error handling fall into one "gap" (254 ms seen).
- **Fixed extras included:** RX_DISPATCH and RELAY include the 10 µs test-point pulse.
- **Weak sample:** TEST_BTN has only 3–50 presses.

Assumptions.

1. **Receive events (catalog maxima).**
   - Each received frame costs C_ev; TX-done for the zone's own frames is inside DRV_FRAME.
   - Per 20 ms there are ≤ 24 frames on the two buses: N1's 4 + N2's 3 + N3's 2 per bus, plus ≤ 6 relayed copies.
   - So **N2 receives ≤ 12 and N3 ≤ 20**, spaced ≥ 222 µs apart per bus (the shortest DLC-8 frame).
   - Event count in a window w: `E(w) = min(2⌈w/222⌉, k⌈w/20 ms⌉)`.
2. **Blocking B (measured).**
   - **STATE_SPI:** the status print's controller reads run with the scheduler locked.
   - `motor_lock` **on N3** (priority inheritance): bounded by HB_MON's whole C, 232 µs.
3. **ISRs:** they only give semaphores, and their time is already inside the measured C values.
4. **Frame-triggered tasks:** R is measured from the end of the triggering frame, so it includes that frame's CAN_RX.

`R_i = C_i + B_i + E(R_i)·C_ev + N_tx(R_i)·C_CAN_TX [+ N_relay·C_RELAY on N2] + Σ_hp ⌈R_i/T_j⌉·C_j`

N2 (steering) (C, B in µs; R in ms):

Task	Prio	C	T	B	D	R	Meets?
CAN_RX (per frame: 680.5 + 50.9)	coop 2	731.4	222 µs/bus	118.5	0.444 (2 RX buffers)	8.90	**No:** a burst can overflow the RX buffers; the other bus's copy covers
CAN_TX ×2	0	155.8	per frame	118.5	20	9.21	Yes
RELAY	1	86.8	≤ 6 / 20 ms	118.5	20	10.85	Yes
STEER	2	37.8	20 ms	118.5	48.3	11.32	Yes
BLINK_F	3	29.2	20 ms	118.5	1 (sync)	11.35	**No**
TEST_BTN	4	65.7	50 ms	118.5	10	11.42	**No** (measured 20 µs)
HB_MON	6	120.1	10 ms	118.5	10	11.54	**No:** one poll late; error chain still meets
HB_TX	7	64.6	20 ms	118.5	20	11.72	Yes
FORCE_TX	8	64.7	50 ms	118.5	50	11.79	Yes

N3 (drivetrain):

Task	Prio	C	T	B	D	R	Meets?
CAN_RX (per frame: 452.9 + 50.1)	coop 2	503.0	222 µs/bus	120.5	0.444	10.18	**No** (as N2)
CAN_TX ×2	0	162.5	per frame	120.5	20	10.51	Yes
DRIVE	2	95.8	20 ms	352.5	0.25 (2 ms − R_CTRL)	11.16	**No** (chain, 3.3)
BLINK_R	3	39.8	20 ms	352.5	1 (sync)	11.20	**No**
TEST_BTN	4	127.9	50 ms	352.5	10	11.33	**No**
PID	5	122.9	10 ms	352.5	10	11.45	**No:** up to 1.5 ms late; uses the real elapsed dt
HB_MON	6	232.0	10 ms	120.5	10	11.57	**No:** one poll late
HB_TX	7	65.2	20 ms	120.5	20	11.87	Yes

Why tasks miss. The driver's per-frame cost (503–731 µs) exceeds the 222 µs frame spacing, and two buses feed each zone. In a worst-case burst the driver threads alone need more than the whole CPU, so every task waits for the burst to be served: about 9–10 ms. Long-run CPU load is still about 60 %.

Fixed because of these measurements. The 1 s status print ran for up to 1.2 ms at Zephyr's default main priority 0, above STEER/DRIVE.

- `main` now drops to the lowest priority after init.
- Its SPI reads run with the scheduler locked. The SPI lock is a `k_sem` with no priority inheritance, so a preempted holder would otherwise invert priorities.
- N3 prints only after releasing `motor_lock`.

B.3 CAN message response times

Equations: `w = B + Σ_hp ⌈(w + J_j + 2 µs)/T_j⌉·C`, then `R = J + w + C`.

Inputs:

- **C = 270 µs:** DLC 8 with worst-case stuffing (assumed, computed).
- **hp:** lower IDs, plus other streams with the **same** ID (relayed copies, other zones' FAULT), counted conservatively.
- **B:** 270 µs for a lower frame on the wire, plus local single-buffer blocking:
  - STM32: one of the sender's own lower frames already in the MCP2515's single TX buffer.
  - Pi: up to all of N1's other frames ahead in the FIFO qdisc (assumed).
- **J:**
  - N1 messages: J_PI (measured; assumed equal for HB_N1, BLINK and FAULT, which come from the same thread).
  - Zone messages: the R of the sending task.
  - Relayed copies: R_CTRL + R_RELAY = 12.6 ms.

The table is for the busier bus, which carries all 6 relayed copies; the other bus is a subset of it.

ID	Message	J	B	w	R	D	Meets?
0x010	CTRL (N1)	0.13	1.08	1.35	**1.75**	2	Yes; chain in 3.3
0x010	CTRL' (relay)	12.6	0.54	0.81	13.7	—	chain in 3.3
0x020	FAULT N1 / N2 / N3	0.13 / 11.54 / 11.57	0.81 / 0.54 / 0.54	2.43 / 2.16 / 2.16	2.83 / 14.0 / 14.0	100	Yes
0x020	FAULT N1' / N3' (relay)	12.6	0.54	2.16	15.0	100	Yes
0x100	HB_N1 / HB_N1'	0.13 / 12.6	0.54	2.70	3.10 / 15.6	20	Yes
0x101	HB_N2	11.72	0.54	2.97	15.0	20	Yes
0x102	HB_N3 / HB_N3'	11.87 / 12.6	0.27 / 0.54	3.24 / 3.51	15.4 / 16.4	20	Yes (3.6 ms slack)
0x200	BLINK / BLINK'	0.13 / 12.6	0.27 / 0.54	3.78 / 4.05	4.18 / 16.9	100	Yes
0x300	FORCE	11.79	0	3.78	15.8	50	Yes

All times are in ms.

Utilization (R4.6):

- Busier bus: 14 × 270/20 000 + 270/50 000 = **19.4 %**.

- Other bus: **11.3 %**.
- Both are under the 50 % limit.

Same-ID frames (assumption): we assume two same-ID frames never start arbitration together. A relayed copy trails its original by at least one relay delay (≥ 474 µs, photo 4), and FAULTs are rare. TEC and REC stayed 0 in every run.

B.4 End-to-end chains

Bound: the sum of the stage bounds above. **Measured:** the maximum of repeated scope captures. Photos 1–5 were taken without `--rt`, so they are conservative.

Chain	Bound (stages, ms)	Bound	Measured max	Limit	Verdict
Brake (R2.2)	CTRL 1.75 + DRIVE 11.16	12.9 ms	**940 µs** (photo 1, 15 runs)	2 ms	**No by analysis; yes as measured**
Throttle (R2.1)	same	12.9 ms	**626 µs** (photo 2, 10 runs)	2 ms	**No by analysis; yes as measured**
Brake via relay (c)	CTRL 1.75 + RELAY 10.85 + CTRL' 1.08 + DRIVE 11.16	24.8 ms	**1.1 ms** (photo 3, 10 runs)	2 ms	**No by analysis; yes as measured.** Degraded latency accepted in (c) (4.3)
Steering (R2.3)	CTRL 1.75 + STEER 11.32	13.1 ms	—	50 ms	**Yes**
Blinker start (R2.4)	BLINK 4.18 + BLINK_F 11.35 / BLINK_R 11.20	15.5 ms	—	100 ms	**Yes**
Blinker sync (R2.4)	max local R 11.35 + skew between bus copies 3.91	15.3 ms	**768 µs** (photo 5, 10 edges)	1 ms	**No by analysis; yes as measured** (232 µs margin)
Local fail (R2.5)	TEST_BTN R	11.4 ms	**20 µs** N2 (photo 12)	10 ms	**No by analysis; yes as measured**
Error, test button (R2.5)	BTN 11.42 + HB_MON 11.54 + FAULT 2.43 + HB_MON 11.57	37.0 ms	**1.1 ms** N2 → N3 (photo 13)	100 ms	**Yes**
Error, zone silent (R4.5)	HB R 15.4 + 60 timeout + 10 poll + HB_MON 11.54	96.9 ms	68.4 ms isolation (photo 14)	100 ms	**Yes**

Why the analysis and the measurements disagree.

- **Cause:** every failing bound comes from one term, the driver burst from 3.1.
- **Why the captures missed it:** the burst needs CTRL to line up with all the heartbeats, BLINK, FAULTs and relayed copies. These are 20 ms streams on three independent clocks. The captures saw normal phasing, with one or two competing frames.
- **Why it still counts:** clock drift means that alignment eventually happens, so the measurements show normal behaviour but don't guarantee the deadline. Even without FAULT traffic, DRIVE's bound is 8.1 ms.

Remedies (not implemented; we kept Zephyr's MCP2515 driver):

1. **Cut the driver's per-frame cost.** Use a lean receive path (READ RX BUFFER on RXnIF), SPI DMA, or hardware acceptance filters. This is the dominant term.
2. **Relay only after a bus is seen degraded.** This removes 6 received frames per 20 ms, but adds the 60 ms detection time to the first relayed frame.
3. **Priority qdisc on the Pi,** with CTRL in the top band. R_CTRL would drop from 1.75 to 0.94 ms.

B.5 Dual bus and relay, with measured WCETs

B.5.1 Hot/parallel transmit

What the code does:

- **Send:** each message is queued on A and then B, with one sequence number (`can_link_send`).
- **Receive:** one dispatch path keeps the first copy by `(ID, seq)`. FAULT gets one dedup slot per sender.
- **Cost:** every message is 2 CAN_TX jobs and 2 receive events, and the second event still pays C_ev. Section 3 already charges both copies, so its tables are the 4.1 re-analysis.

The chains get no worse, because each receiver acts on the first copy and the duplicate only adds one interference term. That assumes:

- the two copies leave together (the Pi writes A then B inside the measured 126.5 µs stage);
- dropping a duplicate is cheap: a table lookup inside RX_DISPATCH, ≤ 51 µs.

B.5.2 Relay design

Question	Decision	Why
Who relays?	Only N2	In the handout's (c) it is the only node on both buses. See the pull list in 4.3 for the cost.
When?	Always, with no detection	Waiting for detection would cut CTRL (brakes included) for 60–70 ms. Costs bus time: worst bus 11.3 → 19.4 %.
What?	The first copy of every N1/N3 frame (CTRL, BLINK, HB_N1, HB_N3, FAULT), onto the other bus	N2's own frames are already on both buses.
Loops?	Relayed flag 0x10 in byte 0; N2 never relays a flagged frame	The sequence number is kept, so receivers dedup with their existing state. The CTRL CRC masks the flags.
Priority?	RELAY = 1: above STEER and all app tasks, below CAN_TX and the driver threads	It's on the brake path. The receive hook only queues (16 deep). TX queues are sorted by ID, so a relayed CTRL overtakes waiting HB/FORCE.
Cost to N2	≤ 6 × 86.8 µs + 6 CAN_TX per 20 ms	Already in N2's table. STEER still meets.

B.5.3 Re-analysis for scenario (c): N1 off B, N3 off A

Traffic: each bus carries 9 unique frames per 20 ms, giving **11.3 % utilization**.

- **Bus A:** N1's 4 + N2's 3 + N3's 2 relayed.
- **Bus B:** N2's 3 + N3's 2 + N1's 4 relayed.
- **Receivers:** N2 receives 6 frames and N3 receives 7, all on B.

Assumption: N3's unacknowledged bus A sends are counted as CAN_TX jobs, but the unplugged module's error interrupts aren't modelled. The relayed chain measurement (photo 3) covers them.

N2 task	R (ms)	Meets?	N3 task	R (ms)	Meets?
RELAY / STEER	6.46 / 6.94	Yes / Yes	DRIVE	4.62	chain (No)
BLINK_F	6.96	No (sync)	BLINK_R	4.66	No (sync)

N2 task	R (ms)	Meets?	N3 task	R (ms)	Meets?
TEST_BTN	7.03	**Yes**	TEST_BTN / PID / HB_MON	4.79 / 4.91 / 4.91	**Yes**
HB_MON / HB_TX / FORCE_TX	7.15 / 7.21 / 7.28	Yes	HB_TX	4.98	Yes

Message R (ms)	Bus A	Bus B	D	Meets?
CTRL / CTRL'	1.48 (J 0.13)	8.75 (J 7.94 = 1.48 + RELAY 6.46)	chain	chain
FAULT	N1 2.02, N2 8.77, N3' 15.2	N1' 9.56, N2 8.77, N3 6.53	100	Yes
HB_N1 / HB_N2 / HB_N3	2.02 / 9.37 / 16.0 (relayed)	9.83 (relayed) / 9.37 / 7.14	20	Yes
BLINK / FORCE	2.56 / 9.71	10.6 (relayed) / 9.71	100 / 50	Yes

Chain in (c)	Stages (ms)	Bound	Measured	Limit	Verdict
Brake / throttle (relayed)	CTRL 1.48 + RELAY 6.46 + CTRL' 0.81 + DRIVE 4.62	13.4 ms	**1.1 ms** (photo 3)	2 ms	**No by analysis; yes as measured**
Steering	CTRL 1.48 + STEER 6.94	8.4 ms	—	50 ms	Yes
Blinker start, front / rear	via A / via relay	9.5 / 16.4 ms	—	100 ms	Yes
Blinker sync	RELAY 6.46 + BLINK' 2.70 + BLINK_R 4.66	13.8 ms	—	1 ms	**No by analysis**
Error: N2 button / N1 dies	7.03 + 7.15 + 1.62 + 4.91 / HB_N1' 9.83 + 60 + 10 + 4.91	20.7 / 84.7 ms	—	100 ms	Yes

Verdict per chain:

- **Brake / throttle:** fails by analysis, measures 1.1 ms (retaken after the bus B fix, same result).
  - What changed: an always-on relay, RELAY above every app task, and ID-sorted TX queues.
  - What's left is the driver cost from 3.1. We take the handout's option to **accept documented degraded latency in (c)**, which is a double fault.
- **Steering and blinker start:** meet with wide margin.
- **Blinker sync:** N3 can only act on N2's relayed copy, so even the typical offset is the relay delay (about 0.5–0.8 ms).
  - Remedy: N2 delays its own edge by the relay latency while it relays.
  - Accepted as degraded in (c).
- **Error propagation:** meets.
  - Without the relay, N3 would declare N1 failed after 60–70 ms. The relay keeps N3 NORMAL; we measured `HB N1:-B` in (c).
  - Relayed HB_N1 copies reach N3 10.4–29.6 ms apart. That's outside 18–22 ms (the ±10 % rule applies at the sender, N1, which still sends every 20 ms), but well inside the 60 ms missed-heartbeat limit.

Which jumper pulls need a relay:

Fault	Relay needed?	Our system
One node off one bus (a/b), two nodes off the same bus, whole bus (e)	No	Fully operational (tested)
N1 and N3 off opposite buses (handout (c))	Yes, via N2	**Fully operational** (tested)
N2 and N3 off opposite buses (Lab 1's R4.8(c) example)	Yes, via N1	**Not supported:** system ERROR
N1 and N2 off opposite buses	Yes, via N3	**Not supported:** system ERROR

Fault	Relay needed?	Our system
One node off both buses (d)	No relay helps	Node fail-safe; others ERROR (tested)

Remedy (not implemented): run the same relay rule on **every** node that is on both buses. It stays loop-free because flagged frames are never forwarded, at the cost of more bus load.